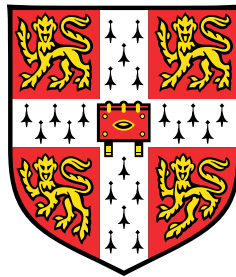


Performing Memory-Intensive Tasks With Test-Time Training



Leann Alhashishi

Department of Engineering
University of Cambridge

This dissertation is submitted for the degree of
Master of Philosophy in Machine Learning and Machine Intelligence

Clare Hall

August 2026

I would like to dedicate this thesis to my mother, Mona, who's been supporting me from thousands of miles away.

Acknowledgements

I would like to thank my supervisors, Dr. Sebastian W. Pattinson and Christos Margadji, for their support throughout this project. I'm grateful to Dr. Pattinson for his ongoing feedback, and to Christos for his insight and guidance, as our weekly discussions have been invaluable to this work.

I am grateful to Rhea, who has become my favorite person over the course of this year, for believing in me from the very first day of this project.

I would also like to thank my mother, Mona, for encouraging me to continue my education, and both her and my siblings – Leen, Yasmeen, and Sultan – for always being there to support me.

Finally, thank you to Saloni, Poulomi, Ishaanvi, and Bermet for their friendship and all the wonderful memories during our time at Cambridge.

Abstract

Temporal reasoning over video requires access to information that is often distributed unevenly across time, yet most vision-language models either (a) store raw frames and retrieve them at query time, or (b) encode video into activations that cannot be queried after the forward pass ends. This thesis investigates a third alternative of encoding video episodes directly into the weight space of specialized neural networks, then projecting those weights into a large language model for downstream reasoning. The central hypothesis is that neural implicit representation weights constitute a semantically structured memory representation sufficient for non-Markovian downstream reasoning, where relevant information is spread across time rather than concentrated in any single frame.

To test this, we propose a pipeline using Neural Implicit Representations (NIRs) fitted to individual robot manipulation videos from a shared meta-initialization learned with MAML, enabling rapid per-episode adaptation and encoding each episode within the network parameters. A trained projector maps the resulting weights into the embedding space of a large language model (TinyLlama-1.1B, Pythia), which then answers multiple-choice temporal reasoning questions and continuous action decoding, such as the task being performed and next state of the robotic arm, without storing or retrieving video frames at inference time.

We evaluate this pipeline on RoboMME, a robot manipulation dataset of memory-intensive tasks such as retrospective counting, state-change tracking of the robot, and subgoal identification. Linear probing directly on the NIR weights shows that memory-intensive content is linearly decodable at up to $3.7\times$ its chance floor, indicating that semantically meaningful information is already present in weight space prior to any language-model decoding. End-to-end QA with TinyLlama-1.1B exceeds a chance baseline by +16.1 percentage points on action selection, 19.1 points on gripper state, and 9.5 points on subgoal identification. The same weight deltas, projected into a pretrained vision-language-action backbone, additionally support continuous action decoding, predicting the next joint command at a mean R^2 of 0.51 with 81.4% gripper sign accuracy, again without frame access.

These results demonstrate that neural implicit representation weights can serve as a queryable memory representation, capable of supporting both language-based reasoning and continuous action decoding, without frame retrieval or explicit memory augmentation.

Table of contents

List of figures	ix
List of tables	xiv
Nomenclature	xvi
1 Introduction	1
1.1 Motivation	1
1.2 Our Contributions	2
1.3 Thesis Outline	3
2 Background	5
2.1 Memory Representations in Machine Learning	5
2.2 Neural Implicit Representations	7
2.3 Learning in Weight Space	10
2.4 Language Models Over Weight Representations	12
3 Encoding Episodes as Neural Implicit Representations	14
3.1 Problem Formulation and Notation	14
3.2 Methodology	16
3.2.1 Task and Dataset	16
3.2.2 Encoding Video Episodes as Neural Implicit Representations	18
3.2.3 Meta-Learning a Shared Coordinate System	19
3.2.4 Per-Episode NIR Fitting	20
3.3 Experiment Design	20
3.3.1 Data Splits and Episode Sampling	20
3.3.2 NIR’s Memory-Intensive Labels	21
3.3.3 NIR Flavor Configurations	22
3.4 NIR Representation Results	23

3.4.1	Reconstruction Quality	23
3.4.2	Meta-Learned Weight Space	25
3.4.3	Linear Classification Probes	27
4	Projecting NIR Weights into Language Space	29
4.1	Methodology	30
4.1.1	From Weights to Token Sequences	30
4.1.2	Integrating with the Language Model	31
4.1.3	Training the Projector	33
4.1.4	Evaluation via Choice-Scoring	34
4.1.5	End-to-End Inference	34
4.2	Experiment Design	34
4.2.1	Evaluation Protocol	34
4.2.2	Metrics and Reporting	36
4.2.3	Projector Training Configuration	37
4.3	Main Results	37
4.3.1	Question-Answering Results	37
4.3.2	Flavor Ablation Results	39
5	Projecting NIR Weights into Action Space	44
5.1	Task Formulation and Scope	44
5.2	Methodology	45
5.3	Training and Evaluation Protocol	46
5.4	Results	46
6	Discussion and Future Work	50
6.1	Synthesis of Results	51
6.2	Limitations	52
6.3	Future Work	54
	References	55
	Appendix A	59

List of figures

2.1	Neural Implicit Representations. A coordinate-based network f_θ maps input coordinates $\mathbf{x} \in \mathbb{R}^d$, which may be spatial position, time, or viewing direction depending on the signal class, directly to signal values $\mathbf{y} \in \mathbb{R}^k$ such as RGB color, geometric density, or feature vectors. The signal is held in the parameters θ and reconstructed on demand through forward passes.	7
2.2	Language Embedded Radiance Fields (LERF) , adapted from Kerr et al. (2023). A NeRF-style implicit network is extended with output heads predicting DINO and CLIP features at each queried location, supervised through a language-alignment loss during fitting. This is the direct precedent for the feature-supervised NIR configurations employed in this work.	8
2.3	Pixel-wise versus image-wise implicit video representation , adapted from Chen et al. (2021). (a) A pixel-wise representation such as SIREN is queried once per spatiotemporal coordinate (x, y, t) , requiring one forward pass per pixel. (b) NeRV instead maps a frame index t to the complete RGB frame in a single forward pass. (c) Each NeRV block upsamples its input through a convolution, a PixelShuffle layer, and an activation.	9
2.4	Model-Agnostic Meta-Learning , adapted from Finn et al. (2017). The initialization θ is meta-optimized (solid trajectory) such that a small number of task-specific gradient steps (dashed) reaches capable adapted parameters θ_i^* for each task.	11

- 2.5 **LLaVA’s projector architecture**, adapted from H. Liu et al. (2023). A frozen vision encoder extracts image features, mapped by a trained linear projection into token embeddings compatible with the language model. These are spliced alongside the tokenized language instruction and passed to the language model, which attends over both to generate a response. LLaVA trains in two stages: the projector alone, with the language model frozen, followed by joint fine-tuning of the projector and language model together. This is the two-stage scheme this thesis follows, with image features here replaced by fitted NIR weights. 13
- 3.1 **Overall System Pipeline.** This figure illustrates the three stages of the proposed method: (1) Per-episode NIR fitting starting from a shared MAML meta-initialization θ , (2) layer-wise folding and projection of the weights via linear maps (PROJ_i) and pooling into weight-tokens, and (3) QA decoding where an LLM (TinyLlama) conditions on these tokens to output an answer without raw frame access. 15
- 3.2 **Representative frames from the four RoboMME memory suites**, one task per suite (columns), sampled over the course of an episode (rows, top to bottom). Counting tasks (PickXtimes) require tracking how many repetitions of an action have occurred; Permanence tasks (ButtonUnmask) require locating a cube whose color was observed before masking; Reference tasks (PickHighlight) require recalling which cube was indicated earlier in the episode; Imitation tasks (MoveCube) require reproducing a demonstrated manipulation. In each case the queried information is not recoverable from the final frame alone. 17
- 3.3 **QA labels on an example episode.** Frames sampled from a single PickX-times episode up to the cut frame T_{cut} (right), annotated with the per-timestep subgoal. The panel below lists the four QA labels extracted at T_{cut} , which pair with the NIR fitted to exactly these frames. Note that the final frame alone is compatible with multiple subgoal and task-goal instances, so distinguishing them requires information from earlier in the episode. 22
- 3.4 **Qualitative frame fitting quality.** Ground truth (GT) frames (top) and decoded outputs from the fitted NeRV instance (bottom) across selected timesteps of a sample episode of task PickXtimes. Per-frame PSNR values (dB) are annotated in the bottom-left corner. 24
- 3.5 **Post-fitting frame reconstruction across architecture flavors.** A single VideoRepick frame decoded from NIRs fitted under Flavors 1-6, with PSNR (dB) relative to the ground truth frame (top). 24

3.6	Dataset-wide NIR fitting performance. Heatmap of mean PSNR (dB) achieved after fitting NIR instances, by task (x-axis) and flavor (y-axis). . . .	25
3.7	t-SNE of NIR weight space, meta (Flavor 2) versus random initialization (Flavor 1), colored by memory suite , as defined in Table 3.1. Two independently fit t-SNE embeddings (PCA-100, perplexity 30) over a shared subset of 8 tasks (2 per suite: BinFill/PickXtimes, VideoUnmask/ButtonUnmask, Pick-Highlight/VideoRepick, MoveCube/InsertPeg, with 100 samples/task). Axes are in arbitrary t-SNE units.	26
3.8	t-SNE of NIR weight space, organized by task within memory suite , as defined in Table 3.1. The same meta-initialized (Flavor 2) embedding as Figure 3.7, recolored by task, with 100 samples per task. Tasks form adjacent but distinguishable sub-clusters within their suites.	27
4.1	Chapter Overview. Schematic of the projection pipeline: a weight delta Δ (one tensor per NIR layer) is projected tensor-by-tensor into a shared latent width, pooled into a fixed-length token sequence, and spliced into a frozen then fine-tuned language model alongside a natural-language question. The model outputs an answer without ever seeing the original video frames.	30
4.2	Prompt Construction. Shows the substitution of a single <code><point></code> placeholder into an explicit <code><point_start></code> , $N \times$ <code><point_patch></code> , <code><point_end></code> span, the chat-template formatting of the resulting human turn and target answer, and the loss mask applied to answer tokens only.	32
4.3	Choice-scoring at inference. A test item’s candidate answers are each scored by their prior-debiased negative log-likelihood (Eq. 4.4); values are shown beside each candidate, and the prediction is the minimum-scoring choice (highlighted).	35
4.4	Choice-scoring at inference , illustrated on two samples (Flavor 2). Every candidate answer is scored by its prior-debiased negative log-likelihood (Eq. 4.4). Scores are shown beside each candidate and the prediction is the minimum. Left: a subgoal item answered correctly, with the true candidate separated from the distractors by a clear margin. Right: a task_goal item in which all candidates are near-duplicate goal instances of the same task and the debiased scores fail to isolate the true instance.	38

- 4.5 **Flavor Ablation by Question Type (Stage 2 Debiased Accuracy).** Debiased accuracy for each NIR flavor configuration (Table 4.4), shown separately per question type. Dashed lines mark each type’s chance baseline. Bars are the mean across 2 seeds, with whiskers denoting ± 1 standard deviation, and axis ranges differing per panel. Meta-initialized flavors with a pixel head (2, 4, 6) clear the baseline decisively on choice_action, gripper, and subgoal, with the sharpest initialization separation on gripper, where random origins sit at or below the reference. Margins on task_goal are small for every configuration, with only the meta-initialized flavors exceeding the baseline. 40
- 4.6 **Matched noise control by flavor and question type (Stage 2 Debiased Accuracy, Test Split).** Debiased accuracy with real weight deltas (filled markers) against the same checkpoints evaluated with the delta replaced by Gaussian noise passed through the trained projector (open markers), for each NIR flavor of Table 4.4. Connector length is the episode-specific contribution, annotated in points. Dashed lines mark each type’s per-item chance baseline. Meta-initialized flavors with a pixel head (2, 4, 6) fall to or below chance under the substitution, Flavor 1 sits almost exactly on top of itself across both markers on every panel, and Flavor 3 shifts only modestly, so only the former group’s margins depend on episode content. 41
- 5.1 **Chapter Overview.** The projection pipeline of this chapter. Weight delta Δ (one tensor per NIR layer) is projected into $N = 56$ weight tokens and spliced into the pretrained TinyVLA backbone (Pythia-400M) alongside the task instruction, and an action head regresses the eight-dimensional joint_action from the final token’s representation. Stage 1 trains the projector and action head with the backbone frozen; Stage 2 fine-tunes all components jointly. No video frame is seen at any point. 45
- 5.2 **Decoding actions from weight-space memory.** Left: predicted against true actions pooled over the seven continuous joint dimensions, each z-scored by its own target statistics; the dashed line marks perfect prediction and colour indicates prediction density on a log scale. Pooled R^2 over these dimensions is +0.53. Right: complete predicted action profiles (orange) against ground truth (black) for episodes at fixed percentiles of mean absolute prediction error, selected mechanically by that ranking rather than by hand. The gripper dimension is excluded from the pooled panel and is predicted with 81.4% sign accuracy. 47

- 5.3 **Per-dimension predictions on the test split (Stage 2).** For the continuous joints (j0 to j6), predicted against true values in raw units, with the identity line dashed and per-panel R^2 . The gripper command (j7) is binary (± 1), so its panel shows the distribution of the continuous output per true class with the decision threshold dashed; thresholding at zero yields 81.4% sign accuracy. 48
- 5.4 **Random-input control (test split).** Per-dimension R^2 for both stages with real weight tokens (solid) and with Gaussian noise substituted for the weight deltas before projection (hatched), under an identical evaluation. Real tokens yield positive R^2 in every dimension. The noise substitution drives every dimension below zero (Stage 1 mean -0.39 , Stage 2 mean -0.55). 49
- A.1 **Within-task action decoding on the test split (Stage 2).** Mean R^2 computed separately inside each of the sixteen RoboMME tasks, where the instruction is constant and task identity can contribute nothing to the score. All sixteen tasks remain above zero, from roughly 0.19 (InsertPeg) to 0.80 (StopCube). 59

List of tables

3.1	RoboMME task summary with average episode length, sourced from Dai et al. (2026).	16
3.2	QA label types used in this work, drawn from the RoboMME HDF5 schema.	21
3.3	The Six NIR flavor configurations , each fit following Section 3.2.4	22
3.4	Linear Classification Probe on Raw NIR Weights. PCA (100 components) followed by a linear classifier. Overall (%) pools correct predictions across all four question types (weighted by each type’s n). Sample sizes (train/eval): choice_action 923/909, gripper 1000/1000, subgoal 1000/1000, task_goal 705/694. Bold marks the best value per column.	28
4.1	The tensor-shape pipeline from a raw weights to language-model tokens. . .	30
4.2	Hyperparameter settings and optimization configuration for Stage 1 (projector alignment) and Stage 2 (end-to-end fine-tuning).	37
4.3	Main End-to-End QA Performance (Flavor 2: Meta-Learned + Pixel Head). Debiased accuracy (mean $\pm$ std across 2 seeds) after subtracting the LLM prior NLL. Chance is the per-item random-guess baseline. The OVERALL row uses the same item-weighted pooling as OVERALL accuracy. Gain is Stage 2 accuracy minus chance, in percentage points. Stage 2 fine-tuning yields substantial gains on choice_action, gripper, and subgoal. Cohen’s κ is reported for Stage 2.	38

4.4	Systematic Ablation of NIR Configurations (Stage 2 Debiased Accuracy, Test Split). Accuracy is mean $\pm$ std across 2 seeds. Meta-learned initializations (θ) outperform random origins across configurations; auxiliary DINO/CLIP supervision yields no overall gain over pixel-only reconstruction, with Flavor 6 showing a selective advantage on subgoal and task_goal (Section 4.3.2). Cohen’s κ (Landis and Koch, 1977) is reported as an additional chance-corrected agreement measure. Δ_{noise} is the overall accuracy retained over the matched noise control of Section 4.2.2 (control=none minus control=random; mean $\pm$ std of per-seed differences, single seed for Flavor 5): meta-initialized configurations with a pixel head lose 8 to 11 points under the substitution, while the remaining configurations are near unchanged, so only the former’s margins are episodic in origin.	39
4.5	Suite Contribution to Overall Accuracy, Flavors 2 and 6 (Test Split, Control=None). Decomposition of pooled Overall accuracy by suite. Pooled Acc. is item-weighted; Macro Acc. weights each suite’s available qtypes equally, isolating any effect of differing item counts across qtypes (Imitation lacks a constructable task_goal question). The close agreement between Pooled and Macro Acc. for every suite/flavor confirms Imitation’s lower contribution is a genuine consequence of its smaller item count, not a distortion of its per-item quality. Contribution (pp) sums to each flavor’s Overall accuracy.	42
5.1	Offline action decoding on the test split (Flavor 2). Per-dimension R^2 for both stages, with Stage 2 mean absolute error and the target standard deviation in raw units (radians; the gripper command j7 is binary, ± 1). Stage 2 improves on Stage 1 in every dimension. Mean R^2 is the unweighted average over the eight dimensions; bootstrap 95% intervals over test episodes are [0.459, 0.509] for Stage 1 and [0.488, 0.539] for Stage 2.	47

Nomenclature

Roman Symbols

g	Language model conditioning on weight tokens and text
q	Natural-language question or task instruction
$\hat{y}$	Predicted answer
$a, \hat{a}$	True and predicted eight-dimensional joint action
X	Video episode; frames $x_1, \dots, x_T$
T	Episode length in frames
T_{cut}	Truncation (cut) frame at which an NIR is fitted and labels are read
n_{cuts}	Number of truncation points sampled per source episode
$\tilde{c}$	Shared latent width of the projector
t_{pl}	Pooled weight tokens per NIR tensor
n	Number of active NIR weight tensors
N	Total weight tokens, $N = n \cdot t_{\text{pl}}$
d_{hidden}	Hidden dimension of the language model
K_j	Candidate-set size of question item j
m	Margin of the contrastive hinge loss
$\mathcal{L}$	Loss function ($\mathcal{L}_{\text{NeRV}}, \mathcal{L}_{\text{flavor}}, \mathcal{L}_{\text{real}}, \mathcal{L}_{\text{swap}}, \mathcal{L}_{\text{contrastive}}$)
R^2	Coefficient of determination

K Inner-loop gradient steps (MAML)

M Outer meta-training steps (MAML)

Greek Symbols

θ Shared meta-learned NIR initialization

ϕ Per-episode fitted NIR parameters

Δ Episode weight delta, $\Delta = \phi - \theta$, the memory representation

ψ Parameters of the trainable projector f_ψ

ω Parameters of the action regression head h_ω

α Per-parameter inner-loop learning-rate vector (MAML)

η Learning rate (e.g. η_{outer})

γ Chronological train fraction within a meta-training episode

λ Loss-weighting coefficient (λ , λ_{dino} , λ_{clip} , λ_c)

κ Cohen’s kappa, chance-corrected agreement

δ Huber loss threshold

σ Standard deviation of the SSIM Gaussian window

Acronyms / Abbreviations

NIR Neural Implicit Representation

MAML Model-Agnostic Meta-Learning

TTT Test-Time Training

LLM Large Language Model

VLA Vision-Language-Action (model)

MLA Memory-Language-Action (model)

QA Question Answering

RAG Retrieval-Augmented Generation

PSNR Peak Signal-to-Noise Ratio

SSIM Structural Similarity Index Measure

MAE Mean Absolute Error

NLL Negative Log-Likelihood

PCA Principal Component Analysis

t-SNE t-distributed Stochastic Neighbor Embedding

DOF Degrees of Freedom

MLP Multi-Layer Perceptron

Chapter 1

Introduction

1.1 Motivation

Machine learning has long drawn architectural inspiration from the human brain. Perceptrons abstracted the neuron into a computational unit (Rosenblatt, 1958), convolutional networks borrowed structural motifs from the visual cortex (LeCun et al., 1989), and attention mechanisms echo, informally, principles of human perception (Bahdanau et al., 2014). With the widespread use of large language and vision models, memory has been approached in a number of different ways, each with its own tradeoffs. We define memory as a persistent representation of information that can be accessed independently of the original input. Such representations particularly matter for temporal reasoning over visual observations, where information relevant to a given decision may be distributed across time rather than concentrated in a singular moment.

Existing memory mechanisms in machine learning models can be broadly divided into three categories. Explicit memory store information as a directly retrievable record: external memory matrices in Neural Turing Machines (Graves et al., 2014), the attached databases in retrieval-augmented generation (RAG) systems (Lewis et al., 2020), and the transformer’s fixed context window (Vaswani et al., 2017), all preserve information by storing the original content or a directly retrievable representation. Implicit, or latent, memory instead stores information in intermediate activations, hidden states, or key-value caches produced during inference, as done in recurrent neural networks and transformers (Hochreiter & Schmidhuber, 1997; Vaswani et al., 2017). These representations disappear once computation ends and therefore cannot serve as persistent memories in the sense used here. More recently, persistent parameter memory has emerged as a third paradigm, in which information is encoded directly into model parameters rather than stored explicitly or retained in transient activations. Titans (Behrouz et al., 2024) demonstrated that continuously updated weights can function as long-term memory

for language modeling, suggesting that neural network parameters themselves can serve as a memory substrate.

This thesis pursues a related but distinct form of persistent parameter memory, for visual observations rather than language. Rather than continually updating the parameters of a single model as in Titans, we encode each incoming video episode into the parameters of its own Neural Implicit Representation (NIR), a compact function whose weights encode the visual content of the episode. Each NIR is fitted from a shared meta-initialization learned using Model-Agnostic Meta-Learning (MAML) (Finn et al., 2017), enabling rapid adaptation from only a small number of gradient steps. Building on Functa (Dupont, Kim, et al., 2022), which showed that fitted implicit neural representation weights can themselves be treated as structured data, we represent each episode by the weight delta between the shared meta-initialization and its adapted parameters. These weight deltas are projected into the embedding space of a large language model, allowing the language model to answer questions over the encoded video episode without retrieving raw frames or maintaining an external memory store.

The central hypothesis of this thesis is that neural implicit representation weights constitute a semantically structured memory representation, sufficient for non-Markovian downstream reasoning where relevant information is distributed unevenly across time rather than concentrated in any single frame.

1.2 Our Contributions

This thesis makes the following contributions:

- A novel pipeline for encoding video episodes into weight space. Neural Implicit Representations (NIRs) are fitted to individual robot manipulation videos from a shared MAML meta-initialization, and the resulting weight deltas are projected as tokens into a large language model (TinyLlama-1.1B) for downstream reasoning without storing or retrieving raw frames at inference time.
- A trained projector that preserves the hierarchical organization of NIR weights while mapping them into the token embedding space of a language model, following the INR2Array scheme of Zhou et al. (2023).
- A systematic ablation across six NIR configurations, varying parameter initialization (random vs. meta-learned) and supervisory signal (pixel reconstruction, DINO features, CLIP features, and combinations thereof). This isolates the individual and combined contributions of meta-learning and self-supervised feature targets to the semantic structure of the resulting weight space.

- Empirical evidence that NIR weights are linearly decodable for memory-intensive task content, prior to any language-model involvement: action selection at 48.3% against a 13.1% chance floor, and subgoal identification at 29.9% against 8.0%, roughly $3.7\times$ chance in both cases.
- An evaluation protocol for weight-space memory on RoboMME that eliminates shortcut solutions endemic to multiple-choice QA over structured labels.
- End-to-end QA results. Choice-scoring evaluation shows action selection at +16.1 percentage points, subgoal identification at +9.5 points, and gripper state at +19.1 points over each question type’s chance baseline.
- An extension of the same weight-space memory representation from language reasoning to continuous action decoding. The NIR weight deltas are projected into a pretrained vision-language-action backbone with its vision input removed, and used to predict the robot’s eight-dimensional next joint command without frame access. The resulting memory-language-action system reaches a mean R^2 of 0.51 across action dimensions and 81.4% gripper sign accuracy on held-out episodes.

1.3 Thesis Outline

The structure of this thesis is as follows:

- **Chapter 2: Background and Related Work.** Provides background on memory in machine learning, covering explicit and implicit memory mechanisms, neural implicit representations and their weight-space properties, and meta-learning and test-time training as mechanisms for fast adaptation, and the projection of non-textual tokens.
- **Chapter 3: Encoding Episodes as NIRs.** Details the proposed pipeline for encoding video episodes into weight space: the RoboMME dataset, the NeRV NIR implementation, the MAML meta-learning setup for the shared initialization, and per-episode fitting. It also describes the experimental design for this stage, and presents reconstruction quality, weight space analysis, and linear probing results.
- **Chapter 4: Projecting Weights into Language Space.** Describes the projection of NIR weights into language-model space: how weights are converted into token sequences, how they are integrated into the language model’s prompt, the two-stage projector training procedure, and the evaluation protocol. It then presents the end-to-end QA performance and NIR flavor ablation results.

-
- **Chapter 5: Projecting Weights into Action Space.** Extends the weight-space memory approach beyond language reasoning by injecting the same projected NIR weight tokens into a pretrained vision-language-action backbone to predict the robot’s continuous next joint command. The chapter describes the resulting memory-language-action architecture and presents the results.
 - **Chapter 6: Discussion and Future Work.** Synthesizes the results of the three previous chapters, discusses their limitations, and concludes with future work.

Chapter 2

Background

This chapter builds the case for treating neural network weights as a memory representation for video: Section 2.1 situates parametric memory against existing storage mechanisms in machine learning, Sections 2.2 and 2.3 establish that Neural Implicit Representation weights can encode video content within a shared representation space, and Section 2.4 reviews how such non-textual representations can be made legible to a language model.

2.1 Memory Representations in Machine Learning

Sequence models must store and retrieve information, but the mechanism by which this happens varies considerably across architectures, and this variation shapes what a model can be said to "remember." Throughout this thesis, memory refers to a persistent representation of information that can be accessed independently of the original input that produced it. Existing approaches can be grouped, roughly, by where information is held:

1. in transient activations,
2. in explicit external storage,
3. or, in more recent work, in the parameters of the network itself.

The first category stores information in the intermediate activations produced during a forward pass. LSTMs (Hochreiter & Schmidhuber, 1997) maintain a hidden state vector H_t that compresses observed sequence history, with gating designed to counteract vanishing gradients. Transformers achieve something structurally similar through its self-attention mechanism and key-value cache, letting each token attend to representations of prior tokens within a fixed context window (Vaswani et al., 2017). What both share is that the stored information is bound

to the current computation, as once the forward pass ends or the context window closes, the representation is gone and cannot be queried on its own terms.

A second line of work externalizes memory instead. Neural Turing Machines (Graves et al., 2014) introduced differentiable read and write heads operating over an external memory matrix, an idea Memory Networks (Weston et al., 2015) then adapted for question answering. Retrieval-Augmented Generation (Lewis et al., 2020) pushed this further into modern language modeling, replacing fixed memory matrices with dense retrieval across arbitrarily large document collections. The common thread here is that storage capacity is decoupled from model capacity, so memory can scale without touching the network itself. The tradeoff is that memory becomes an explicit record that must be retrieved before reasoning can proceed at inference.

A third and more recent paradigm encodes memory directly into the parameters. Titans (Behrouz et al., 2024), for instance, equips a dedicated memory module with the ability to update its own weights during inference via gradient-based learning, of the forms:

$$M_t = (1 - \alpha_t)M_{t-1} + S_t \quad (2.1)$$

$$S_t = \eta_t S_{t-1} - \theta_t \nabla \ell(M_{t-1}; x_t), \quad (2.2)$$

where a meta-learned surprise signal S_t governs which inputs actually get written into memory by combining the immediate gradient (momentary surprise, $\nabla \ell$) with past context momentum ($\eta_t S_{t-1}$). The data-dependent forget gate $\alpha_t \in [0, 1]$ applies adaptive weight decay to clear out stale information. Because information accumulates in the weights rather than in a buffer or a single forward pass, this allows memory that persists across arbitrarily long sequences without the constraint of a fixed context window. GradMem (Kuratov et al., 2026) takes a related route, performing test-time gradient descent on a small set of learnable memory tokens while the underlying language model stays frozen. The result is a compact representation from which later queries can be answered without further access to the original data. Titans optimizes parameters directly, GradMem optimizes memory tokens, yet both point to the same underlying claim: gradient-based optimization at inference time can construct persistent memory for the model.

Two questions remain open, and they motivate the rest of this thesis. First, parameter-memory architectures such as Titans and GradMem have been developed primarily for language sequences, and it is not obvious that the same principle transfers to temporal reasoning over visual information. Second, these methods continually update a single running memory over an incoming stream rather than representing individual experiences as separable “memories”. Whether the contents of a non-Markovian video episode (where relevant information is dis-

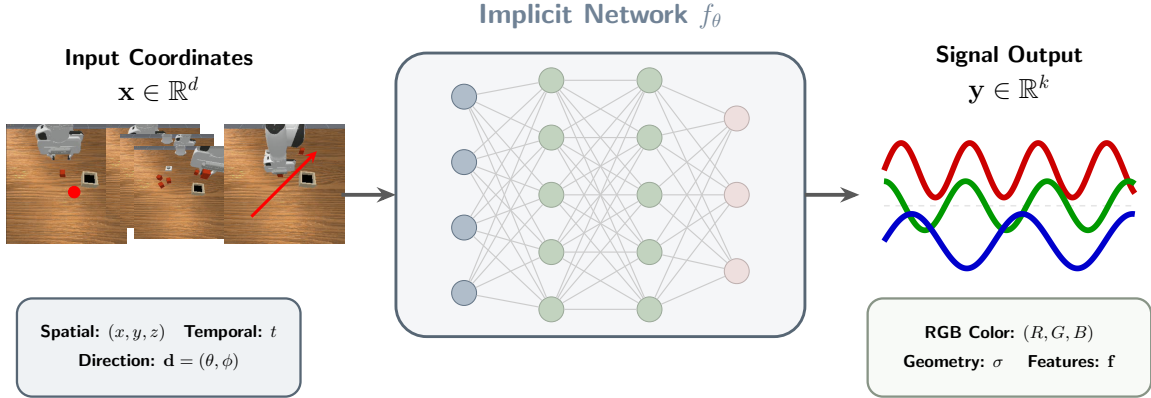


Fig. 2.1 **Neural Implicit Representations.** A coordinate-based network f_θ maps input coordinates $\mathbf{x} \in \mathbb{R}^d$, which may be spatial position, time, or viewing direction depending on the signal class, directly to signal values $\mathbf{y} \in \mathbb{R}^k$ such as RGB color, geometric density, or feature vectors. The signal is held in the parameters θ and reconstructed on demand through forward passes.

tributed unevenly across time rather than concentrated in any single frame) can instead be encoded as its own parameter-space representation, and subsequently reasoned over as such, is the question this thesis takes up. Section 2.2 turns to Neural Implicit Representations as a candidate mechanism for constructing exactly this kind of parameter-space memory.

2.2 Neural Implicit Representations

Section 2.1 argued that model parameters can, in principle, serve as a persistent memory substrate. Left unaddressed was whether parameters can faithfully encode signals as temporally extended and visually rich as video. Neural Implicit Representations (NIRs) are the mechanism this thesis relies on to answer that question. An NIR is a neural network trained to encode a signal as a continuous function, $f(x) \rightarrow y$, mapping input coordinates directly to signal values rather than storing data on a discrete grid (see Figure 2.1). Once trained, the model no longer needs the original image or video frames in storage at all, as the signal is reconstructed on demand through forward passes over the learned parameters. For the memory representation this thesis proposes to be of use, these weights must capture not just perceptual detail but the spatial, semantic, and temporal structure of a video, and the NIR literature demonstrates each of these properties through varying architecture.

Encoding high-frequency visual detail is the most basic requirement. Neural fields built on ReLU activations are known to suffer from spectral bias, which limits how much fine detail they can represent. Sinusoidal Representation Networks (SIREN; Sitzmann, Martel,

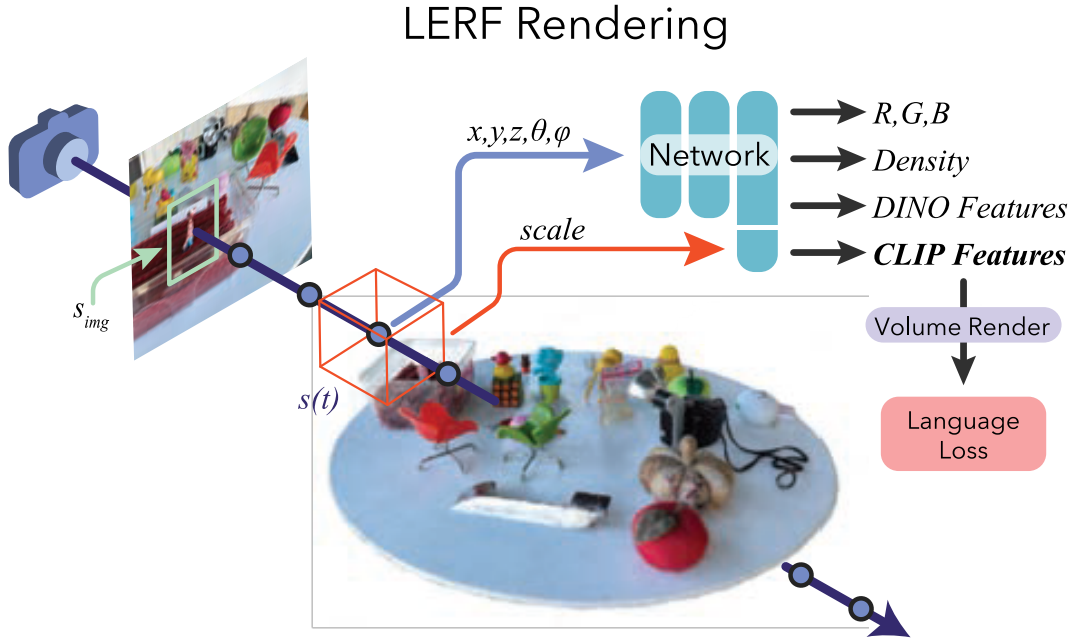


Fig. 2.2 **Language Embedded Radiance Fields (LERF)**, adapted from Kerr et al. (2023). A NeRF-style implicit network is extended with output heads predicting DINO and CLIP features at each queried location, supervised through a language-alignment loss during fitting. This is the direct precedent for the feature-supervised NIR configurations employed in this work.

et al., 2020) address this with periodic sine activations of the form $\phi_i(\mathbf{x}_i) = \sin(\mathbf{W}_i \mathbf{x}_i + \mathbf{b}_i)$, where frequency support is modulated by scaling the input weight matrices. It therefore yields high-fidelity reconstructions of images, objects, and audio. This established the basic feasibility of representing perceptual signals within NIR parameters.

Subsequent work pushed on what could be encoded. Neural Radiance Fields (NeRF; Mildenhall et al., 2020) extended implicit representations from two dimensions into three, mapping spatial coordinates and viewing direction to color and density, $F_\Theta : (\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$, so that scene geometry and appearance could be recovered directly from learned parameters, enabling novel view synthesis. Language Embedded Radiance Fields (LERF; Kerr et al., 2023), as seen in Figure 2.2, built on this by distilling CLIP feature embeddings (Radford et al., 2021) into a NeRF’s weight space. This allowed open-vocabulary semantic queries over the encoded scene, something a purely geometric NeRF cannot support.

Video, being a substantially larger signal than a static image or scene, makes fitting speed a practical bottleneck for these architectures, which are optimized for images and static 3D content. Neural Representations for Videos (NeRV; Chen et al., 2021) sidesteps this by representing video as a function of time t , $f_\theta : t \mapsto v_t \in \mathbb{R}^{H \times W \times 3}$, predicting a full RGB frame v_t from a single temporal coordinate rather than querying spatial locations one at a time, as

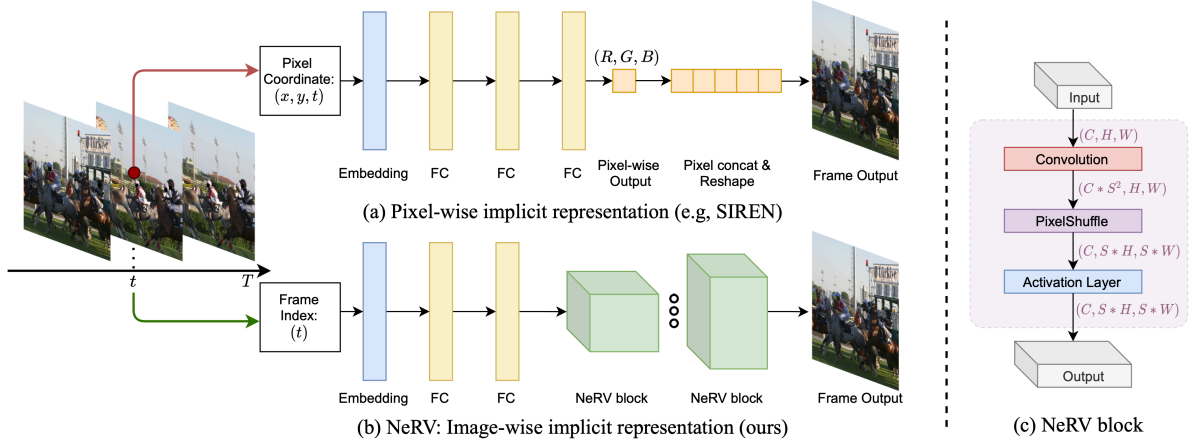


Fig. 2.3 **Pixel-wise versus image-wise implicit video representation**, adapted from Chen et al. (2021). (a) A pixel-wise representation such as SIREN is queried once per spatiotemporal coordinate (x, y, t) , requiring one forward pass per pixel. (b) NeRV instead maps a frame index t to the complete RGB frame in a single forward pass. (c) Each NeRV block upsamples its input through a convolution, a PixelShuffle layer, and an activation.

seen in Figure 2.3. This is considerably faster to fit and reconstruct than pixel-wise NIRs, and it is largely why NeRV has become one of the more widely adopted NIR architectures for video representation and compression (Gwilliam et al., 2025). It is also the implicit representation this thesis builds on.

Later architectures explore, in different ways, how video structure specifically might be encoded in parameter space. ActINR (Kayabasi et al., 2025) extends NeRV by separating shared scene information from frame-specific information, sharing weights across frames while generating frame-dependent biases from a second implicit network. NIRVANA (Maiya et al., 2022) takes a different route, segmenting video into frame groups and fitting a separate NIR to each group, with each new group’s network initialized from the previous group’s weights. The result is an autoregressive trajectory through weight space, and although NIRVANA was proposed for efficient video compression rather than memory, this sequential evolution of parameters bears a close resemblance to the continual accumulation of information that an episodic memory system would need.

Taken together, this group of work shows that NIRs can encode perceptual content, geometric structure, semantics, and temporal evolution. What has not been established is whether the structure encoded into NIR weights has the kind of geometric coherence needed to extract and reason over it after the fact, a question Section 2.3 takes up.

2.3 Learning in Weight Space

Section 2.2 established that NIRs can encode rich visual information in their parameters. Reconstruction fidelity alone, however, does not imply that the resulting parameters form a usable representation space. If independently fitted networks land at arbitrary points in parameter space, two networks encoding near-identical visual content could nonetheless have unrelated weight vectors, in which case comparing or reasoning over weights directly would be ill-posed: coordinate i in one network would bear no necessary relationship to coordinate i in another. Weights-as-memory therefore depends on two conditions holding simultaneously: that fitted weights carry semantic structure, and that this structure sits within a shared coordinate system across instances.

The foundational work in this direction is Functa (Dupont, Kim, et al., 2022), which introduced the concept of “functions as data.” Rather than treating a fitted Neural Implicit Representation solely as a reconstruction model, Functa treats each fitted function as a data point in a learned representation space. Classifiers trained directly on the resulting modulation vectors generalize to unseen instances, meaning semantic relationships in the original data are preserved as geometric relationships in weight space. However, the effect depends on all fitted functions sharing a common origin, so that corresponding coordinates across weight vectors remain comparable. More recent theoretical work grounds this observation more formally, arguing via the Implicit Function Theorem (Qiu et al., 2026) that NIR weights can preserve semantic information about their training data when fitted near a shared initialization rather than arbitrarily far from it.

This shared-origin requirement is what makes meta-learning central to weight-space work. Model-Agnostic Meta-Learning (MAML; Finn et al., 2017) learns an initialization such that a handful of gradient updates on a new task yields capable task-specific parameters, which in the NIR setting offers a way to produce fitted weights that remain comparable across instances: each video-specific solution is reached by adapting from the same starting point rather than from scratch, following Figure 2.4.

COIN++ (Dupont, Loya, et al., 2022) applies this combination of shared initialization and fast per-instance fitting to NIRs specifically, meta-learning a shared base network and representing each new signal through low-dimensional modulations fitted at inference time. It is a close precedent for treating adaptation from a common origin as an encoding mechanism, though COIN++ still uses the adapted representations mainly for compression and reconstruction rather than as memory to be queried downstream. Gielisse and van Gemert (2025) offer further evidence in this direction, showing that meta-learned SIREN initializations combined with a transformer over the resulting weights can outperform permutation-equivariant alternatives on classification.

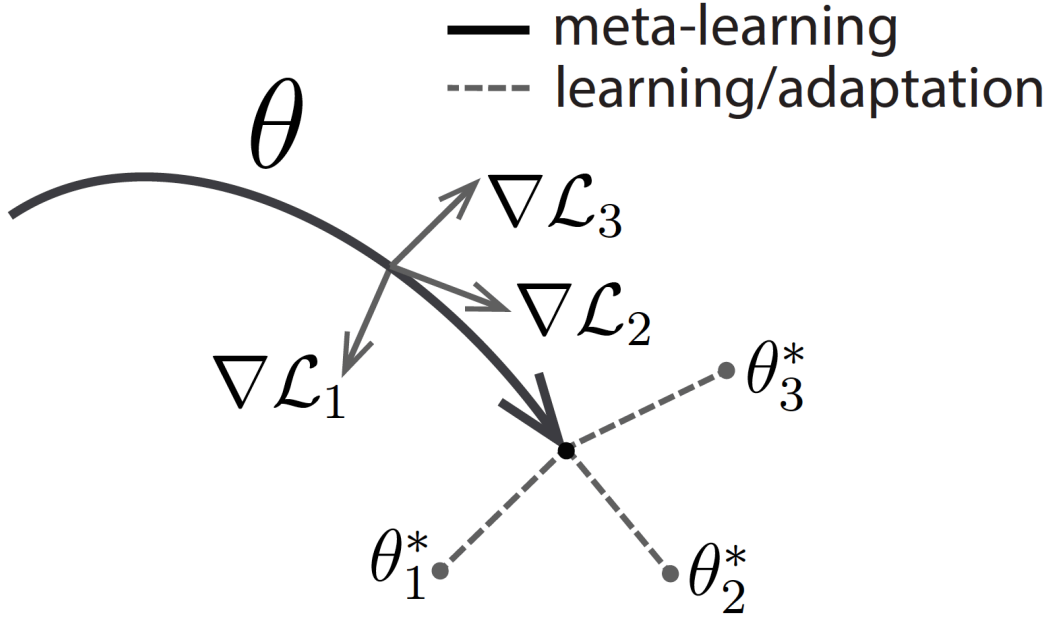


Fig. 2.4 **Model-Agnostic Meta-Learning**, adapted from Finn et al. (2017). The initialization θ is meta-optimized (solid trajectory) such that a small number of task-specific gradient steps (dashed) reaches capable adapted parameters θ_i^* for each task.

Once fitted weights sit in a shared coordinate system, a further question arises: can standard learning algorithms exploit that structure directly? INR2Vec (Luigi et al., 2023) suggests yes, training an encoder over fitted INR weights for 3D shapes can produce embeddings that support retrieval, clustering, and classification. The success of these downstream tasks implies that semantic information is already present in weight space rather than being manufactured entirely by the model sitting downstream of it.

Weight-space representations have, more recently, begun serving as direct inputs to foundation models. LLaNA (Amaduzzi et al., 2024) projects fitted NeRF weights into the embedding space of a frozen large language model, thereby enabling natural-language queries over 3D scenes without rendering or retrieval. This demonstrates that projected NIR weights can support language reasoning, not merely conventional downstream tasks like classification. However, LLaNA’s setting is static: it operates on fitted NeRF weights for 3D scenes, whereas this thesis is concerned with NeRV weight representations fitted to temporally extended video episodes, a less settled problem.

A remaining practical question is whether weight-space representations of this kind can be produced efficiently at inference time. Test-Time Training (Sun et al., 2020) and TTT++ (Y. Liu et al., 2021) show that a small number of gradient updates on a self-supervised objective can adapt a model to individual test instances. TTT on Video Streams (R. Wang et al., 2025)

extends this to video by initializing each frame’s model from the previous frame’s adapted state, which suggests temporal continuity itself carries a useful learning signal.

Together, these strands establish the conditions under which NIR weights function as representations rather than mere means to reconstruction. What remains largely untested is whether these ideas extend to non-Markovian video episodes. This thesis addresses that gap using a meta-learned NeRV initialization to construct a NIR per video episode, then asks whether the resulting weights can function as memory representations for downstream temporal reasoning.

2.4 Language Models Over Weight Representations

Sections 2.2 and 2.3 established the conditions under which NIR weights can serve as structured memory: encoding meaningful visual information, occupying a shared coordinate system, and being producible through efficient adaptation. One question remains before such representations are useful in practice, namely how they can be queried at all. Because the downstream reasoning tasks in this thesis are posed as natural-language questions, the memory representation must ultimately be made legible to a language model. Recent multimodal language models offer a natural mechanism: a trained projector maps non-textual representations into the embedding space of a frozen, pretrained language model, letting the model reason over them without any change to its own parameters.

LLaVA (H. Liu et al., 2023) established this approach for image understanding, where a CLIP vision encoder extracts image features and a learned projection layer maps them into the language model’s token embedding space. The language model then receives projected visual embeddings alongside text tokens and generates a response, as illustrated in Figure 2.5. This therefore allows image-language reasoning to be possible, but with most of the reasoning capacity remaining where it already was, in the pretrained language model. The projector acts only as an interface.

For this thesis, the relevant extension is from visual features to weight-space representations specifically. LLaNA (Amaduzzi et al., 2024) is again the closest precedent, projecting fitted NeRF weights into a frozen language model’s embedding space to support natural-language queries over 3D scenes without rendering or retrieving explicit visual observations. This is evidence that a language model can condition directly on projected NIR weights, as opposed to pixels, image features, or reconstructed views, which is a meaningfully different kind of input.

Existing work currently leaves the setting considered here largely open. LLaNA operates on fitted NeRF weights for static 3D scenes, as well as using full fitted weights rather than adaptation weights from a shared meta-initialization. This thesis instead projects MAML-

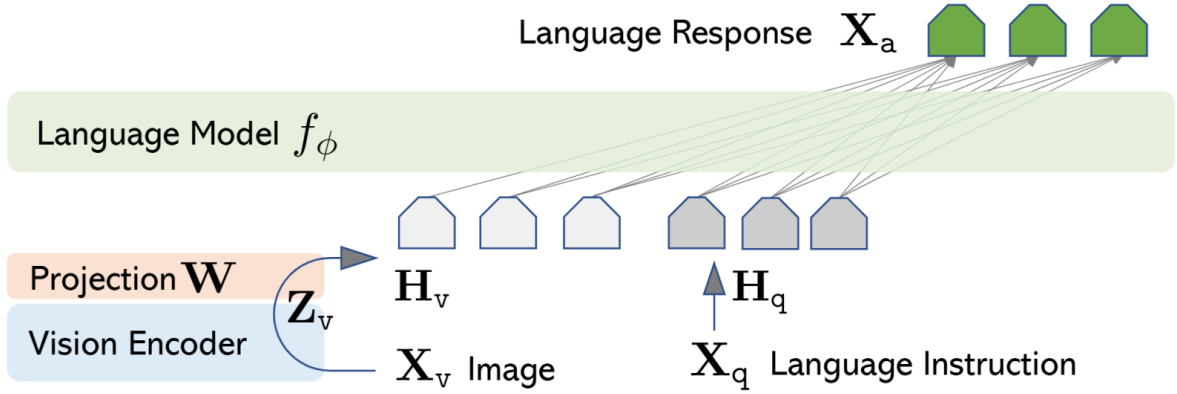


Fig. 2.5 LLaVA’s **projector architecture**, adapted from H. Liu et al. (2023). A frozen vision encoder extracts image features, mapped by a trained linear projection into token embeddings compatible with the language model. These are spliced alongside the tokenized language instruction and passed to the language model, which attends over both to generate a response. LLaVA trains in two stages: the projector alone, with the language model frozen, followed by joint fine-tuning of the projector and language model together. This is the two-stage scheme this thesis follows, with image features here replaced by fitted NIR weights.

adapted NeRV weights. The distinction matters, as what gets projected is not a static scene representation but an episode-specific memory trace.

Prior work has shown language models reasoning over projected image features, video features, and static NIR weights individually. This thesis extends that paradigm to weight-space memories of non-Markovian video episodes, asking whether a language model can answer temporal reasoning questions from projected NIR weights alone, without access to stored frames, reconstructed video, or any explicit retrieval mechanism.

Chapter 3

Encoding Episodes as Neural Implicit Representations

We start by converting each video episode into a Neural Implicit Representation (NIR): a compact network fitted to the episode from a shared, meta-learned initialization, whose weights constitute the episode’s memory representation. This chapter details that process following Figure 3.1, the experimental design, and the resulting reconstruction and weight-space probing results. The subsequent chapter projects these memory representations into a large language model.

3.1 Problem Formulation and Notation

Given a video episode X consisting of a sequence of frames $x_1, \dots, x_T$ and a natural-language question q about the episode, the system produces a predicted answer $\hat{y}$, factorizing this mapping through an explicit weight-space memory representation.

An episode is first encoded into the parameters of a small neural network by fitting it to reconstruct the video. Let ϕ denote the resulting parameter vector and θ a shared, task-agnostic initialization common to all episodes. Because adapted and initialization weights are typically close in magnitude, with the initialization dominating the raw parameter values, we follow Gielisse and van Gemert (2025) in taking the difference between them as the representation rather than the raw fitted weights. The episode’s memory representation is therefore the delta

$$\Delta = \phi - \theta, \tag{3.1}$$

which isolates the change induced by that specific episode from the initialization shared across the dataset. A learned projector f_ψ maps Δ into a sequence of token embeddings compatible

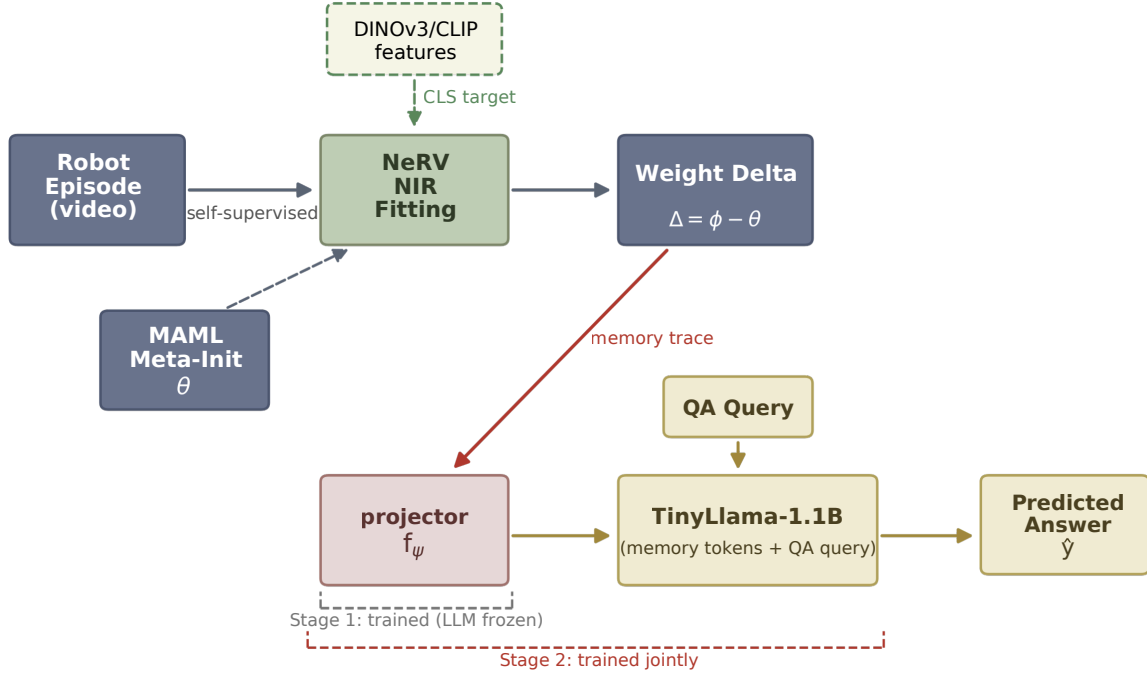


Fig. 3.1 **Overall System Pipeline.** This figure illustrates the three stages of the proposed method: (1) Per-episode NIR fitting starting from a shared MAML meta-initialization θ , (2) layer-wise folding and projection of the weights via linear maps (PROJ_i) and pooling into weight-tokens, and (3) QA decoding where an LLM (TinyLlama) conditions on these tokens to output an answer without raw frame access.

with a language model g . The predicted answer is then

$$\hat{y} = g(f_\psi(\Delta), q), \quad (3.2)$$

where the language model conditions on both the projected weight-tokens and the natural-language question, but has no access to the original frames.

This formulation spans the next two chapters. This chapter addresses the left-hand side: how θ is meta-learned and how each episode's ϕ (and hence Δ) is obtained. Chapter 4 addresses f_ψ and g : how Δ is mapped into language-model token space, how the projector is trained with the language model frozen and then jointly fine-tuned with it, and how the system is evaluated at inference time, with g decoded by scoring each candidate answer's prior-debiased likelihood.

3.2 Methodology

3.2.1 Task and Dataset

All experiments use RoboMME (Dai et al., 2026), a benchmark of question-answering videos built on the ManiSkill simulator (Mu et al., 2021) with a 7-DOF Franka Panda arm. Each video shows the arm performing a tabletop manipulation task, and the accompanying questions probe aspects of the episode that no isolated frame can resolve. In this sense, all 16 task types in RoboMME are non-Markovian: answering correctly at timestep t requires information distributed across the episode. Any model that succeeds must therefore have encoded information beyond the frame it is looking at, which is what makes the benchmark suitable for a memory-focused system.

The 16 task types form four suites, each targeting a distinct form of memory: Counting (temporal), Permanence (spatial), Reference (object), and Imitation (procedural), summarized in Table 3.1. Each episode is stored per-timestep, with fields grouped into episode-level setup and per-timestep obs, action, and info. Figure 3.2 shows representative frames from each suite.

Table 3.1 **RoboMME task summary** with average episode length, sourced from Dai et al. (2026).

Suite	Task	Avg. Steps
Counting	PickXTimes	538
Counting	BinFill	604
Counting	SwingXTimes	435
Counting	StopCube	317
Permanence	VideoUnmask	217
Permanence	ButtonUnmask	267
Permanence	VideoUnmaskSwap	348
Permanence	ButtonUnmaskSwap	400
Reference	PickHighlight	346
Reference	VideoRepick	687
Reference	VideoPlaceButton	974
Reference	VideoPlaceOrder	1134
Imitation	MoveCube	394
Imitation	InsertPeg	479
Imitation	PatternLock	208
Imitation	RouteStick	370

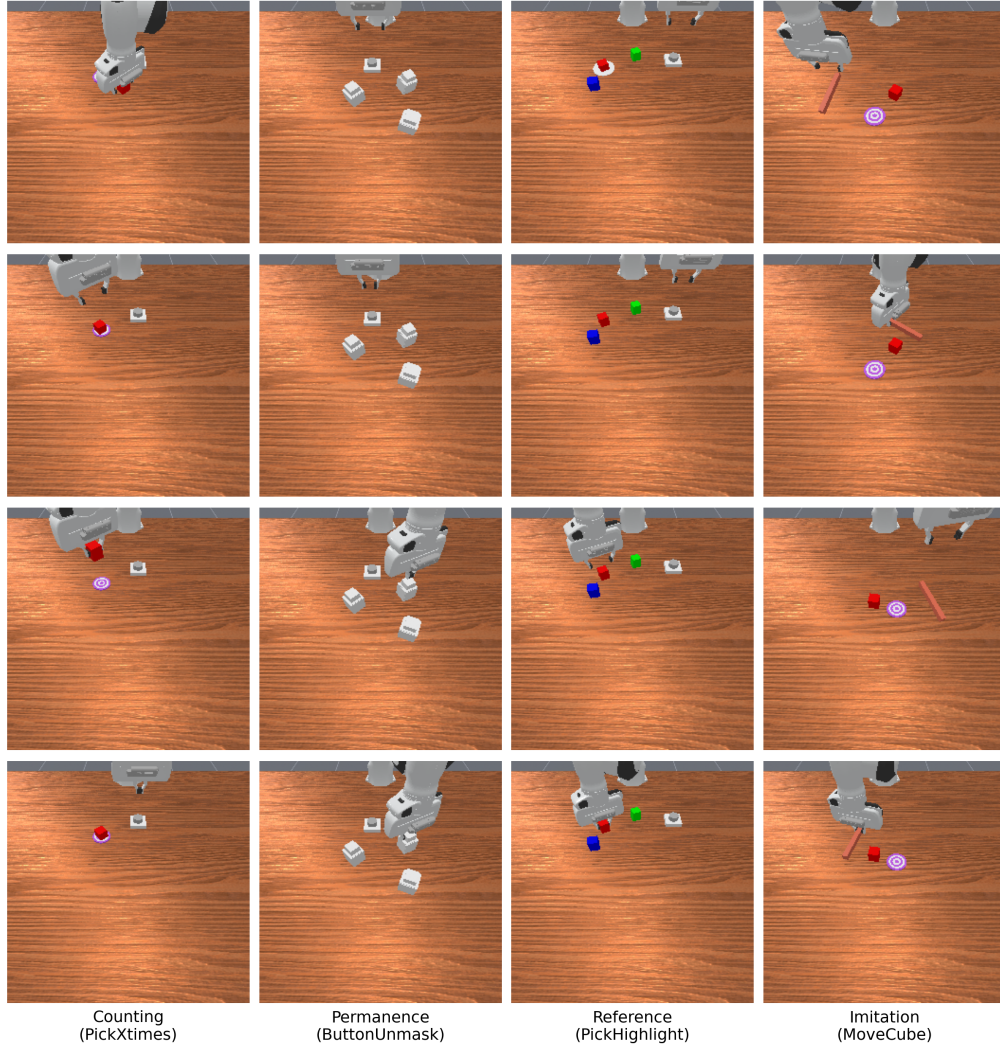


Fig. 3.2 **Representative frames from the four RoboMME memory suites**, one task per suite (columns), sampled over the course of an episode (rows, top to bottom). Counting tasks (PickXtimes) require tracking how many repetitions of an action have occurred; Permanence tasks (ButtonUnmask) require locating a cube whose color was observed before masking; Reference tasks (PickHighlight) require recalling which cube was indicated earlier in the episode; Imitation tasks (MoveCube) require reproducing a demonstrated manipulation. In each case the queried information is not recoverable from the final frame alone.

3.2.2 Encoding Video Episodes as Neural Implicit Representations

Each episode is represented as a Neural Implicit Representation (NIR): a coordinate-based network that maps a scalar timestep to the corresponding video frame. The architecture follows NeRV (Chen et al., 2021), in which a single forward pass produces an entire $H \times W$ frame rather than one pixel per pass, making both fitting and decoding substantially faster than pixel-wise implicit representations. Full architectural specifications follow Appendix A.1 of Chen et al. (2021).

The NIR architecture comprises four sequential components:

- **Positional Encoding:** Lifts the normalized scalar timestep t into a high-frequency sinusoidal embedding $\gamma(t) \in \mathbb{R}^{2L}$ with $L = 80$ and base $b = 1.25$ to prevent spectral bias:

$$\gamma(t) = (\sin(b^i \pi t), \cos(b^i \pi t))_{i=0}^{L-1} \quad (3.3)$$

- **MLP Stem:** A 3-layer MLP with GELU activations (Hendrycks & Gimpel, 2016) that maps $\gamma(t)$ to a spatial feature map of shape $(\text{base_ch}, h_0, w_0)$, where $h_0 = H/2^n$ and $w_0 = W/2^n$ for n upsampling blocks.
- **Upsampling Blocks:** A series of n NeRV blocks that progressively scale up spatial dimensions, each applying a 3×3 convolution, scaling the channels by scale^2 , a PixelShuffle layer doubling H and W (Shi et al., 2016), and a GELU activation to avoid checkerboard artifacts.
- **Output Head:** A final 1×1 convolution compressing the final feature channels down to 3 (RGB) channels, bounded by a sigmoid function to enforce $[0, 1]$ outputs, with bilinear interpolation to exact dimensions (H, W) where needed.

Fitting Objective

Each NIR is fit by minimizing a weighted combination of $\mathcal{L}_1$ reconstruction and structural dissimilarity:

$$\mathcal{L}_{\text{NeRV}} = \lambda \cdot \|\hat{x} - x\|_1 + (1 - \lambda) \cdot (1 - \text{SSIM}(\hat{x}, x)), \quad (3.4)$$

with $\lambda = 0.7$ (Chen et al., 2021). SSIM is computed with an 11×11 Gaussian window ($\sigma = 1.5$) applied per channel via grouped convolution (Z. Wang et al., 2004). The $\mathcal{L}_1$ term encourages pixel-accurate reconstruction, while the SSIM term preserves local structural patterns such as edges and textures that $\mathcal{L}_1$ alone can blur.

3.2.3 Meta-Learning a Shared Coordinate System

Section 2.3 established why NIR weights require a shared coordinate system to function as a comparable representation space, and why Functa (Dupont, Kim, et al., 2022) and subsequent implicit-function-theorem arguments point to a common fitting origin as the resolution. Accordingly, every NIR in this work is fit from a common initialization θ , learned via Model-Agnostic Meta-Learning (Finn et al., 2017), which optimizes θ so that a small number of gradient steps from θ produces a good per-episode fit on any episode from the training distribution. Meta-training is a bilevel optimization. An inner loop adapts θ to a specific episode over a fixed number of gradient steps, and an outer loop updates θ so that this adaptation is effective across episodes on average. Algorithm 1 gives the full procedure.

Algorithm 1 MAML meta-training for the shared NIR initialization

Require: Meta-training episodes $\{\mathcal{E}_i\}_{i=1}^N$ (full-batch, no per-step sampling), temporal stride $s = 4$, inner steps $K = 3$, outer steps $M = 1,000$, outer learning rate $\eta_{\text{outer}} = 0.001$, chronological train fraction $\gamma = 0.7$, initial per-parameter inner learning rate vector $\alpha = 10^{-3} \cdot \mathbf{1}$.

Ensure: θ, α .

```

1: for outer step  $m = 1$  to  $M$  do
2:   for each episode  $\mathcal{E}_i \in \{\mathcal{E}_i\}_{i=1}^N$  do
3:      $F_{\text{train}} \leftarrow \mathcal{E}_i[1 : \lfloor \gamma T_i \rfloor]$ ,  $F_{\text{test}} \leftarrow \mathcal{E}_i[\lfloor \gamma T_i \rfloor + 1 : T_i]$ 
4:      $\phi_0 \leftarrow \theta$ 
5:     for  $k = 0$  to  $K - 1$  do
6:        $\phi_{k+1} \leftarrow \phi_k - \alpha \odot \nabla_{\phi_k} \mathcal{L}_{\text{NeRV}}(\phi_k; F_{\text{train}})$ 
7:     end for
8:      $\ell_i \leftarrow \mathcal{L}_{\text{NeRV}}(\phi_K; F_{\text{test}})$ 
9:   end for
10:   $\mathcal{L}_{\text{outer}} \leftarrow \frac{1}{N} \sum_{i=1}^N \ell_i$ 
11:  Clip  $\nabla_{(\theta, \alpha)} \mathcal{L}_{\text{outer}}$  to max norm 1.0
12:   $(\theta, \alpha) \leftarrow (\theta, \alpha) - \eta_{\text{outer}} \cdot \nabla_{(\theta, \alpha)} \mathcal{L}_{\text{outer}}$ 
13: end for

```

The inner-loop learning rate α is a per-parameter vector learned jointly with θ , following the MetaSDF formulation of Sitzmann, Chan, et al. (2020), so step size adapts on a per-weight basis rather than through a single global rate.

Scope of meta-training

Meta-training uses a subset of five task types (BinFill, PickXtimes, SwingXtimes, VideoUnmask, PatternLock; 20 episodes each) rather than the full 16-task corpus, following MAML’s premise that a shared initialization learned on a subset of tasks generalizes to unseen ones (Finn et al., 2017). The learned θ transfers to episodes from all 16 tasks at inference (Section 3.2.4). This meta-training split is distinct from the corpus-wide train/validation/test partition used for all downstream evaluation. Within each meta-training episode, MAML’s own inner/outer loop (Algorithm 1) uses a chronological 70/30 split of that episode’s own frames: the first 70% ($\gamma = 0.7$) as F_{train} for inner-loop adaptation, and the remainder as F_{test} for the outer-loop loss.

3.2.4 Per-Episode NIR Fitting

Per-episode fitting adapts θ to a single episode via 500 steps of full-batch Adam ($\eta = 5 \times 10^{-4}$) (Kingma & Ba, 2017) on the reconstruction objective of Section 3.2.2, with no held-out split and no meta-optimization. The episode’s memory representation is the resulting delta $\Delta = \phi_{\text{fit}} - \theta$ of Section 3.1, and it is Δ rather than the raw weights that the projector in Section 4.1.3 is trained on.

3.3 Experiment Design

3.3.1 Data Splits and Episode Sampling

RoboMME’s 16 task types each contribute 100 source episodes, giving a base corpus of 1,600 episodes split 70/15/15 into train, validation, and test partitions. A single episode does not produce only a single NIR: for each episode, $n_{\text{cuts}} = 10$ random truncation points T_{cut} are drawn from its timeline (minimum offset `min_frames` = 10, seeded deterministically per task and episode). An NIR is fit independently to the frames up to each T_{cut} , and the label fields are extracted at that last frame. Each source episode therefore supplies multiple temporally distinct NIRs.

Table 3.2 lists the fields read out at each cut, which are the raw sources of every question type used downstream. Across all tasks, episodes, and cuts, this yields up to 96,000 stored weights ($16 \text{ tasks} \times 100 \text{ episodes} \times 10 \text{ cuts} \times 6 \text{ flavors}$), with frames rendered at $H = W = 64\text{px}$ throughout.

3.3.2 NIR’s Memory-Intensive Labels

Since the model answering a downstream question receives only Δ , and never the frames, a label tests memory only to the extent that answering it requires information beyond what the final observed frame contains, the non-Markovian property defined previously. The four labels sit at different points along that axis, characterized here in order of increasing dependence on memory. Figure 3.3 illustrates all four on a single episode.

gripper. A binary label from `obs/is_gripper_close`, recording whether the gripper is closed at the cut frame. Can be determined from the final frame alone.

choice_action. A multiple-choice label over a candidate set of natural-language actions (`setup/available_multi_choices`), stored as a choice letter in `action/choice_action` and resolved to its action string. It names the atomic action alone ("*pick up the cube*") without count or instance information.

subgoal. A categorical label from `info/simple_subgoal`, annotating the phase the robot is currently executing. Most importantly, phases recur within an episode: in a representative VideoPlaceButton episode, "*pick up the cube*" occurs four separate times, and in the Counting suite the vocabulary is count-indexed ("*pick up the blue cube for the **third** time*"). A snapshot of the final frame is therefore consistent with several distinct subgoal labels, and disambiguating them requires progress information accumulated over the episode.

task_goal. A categorical label from `setup/task_goal`: the natural-language instruction governing the episode, fixed at its start. Within a task, goal instances are near-duplicates differing only in a single attribute, such as the manipulated color, a repetition count, or an

Table 3.2 **QA label types** used in this work, drawn from the RoboMME HDF5 schema.

Label	HDF5 source	Type	Description
task_goal	setup/task_goal	categorical	Language-described goal instance for the episode
subgoal	info/simple_subgoal	categorical	Current subgoal text (planner view)
gripper	obs/is_gripper_close	binary	Whether the gripper is open or closed
choice_action	action/choice_action	categorical	Native multi-choice action selection

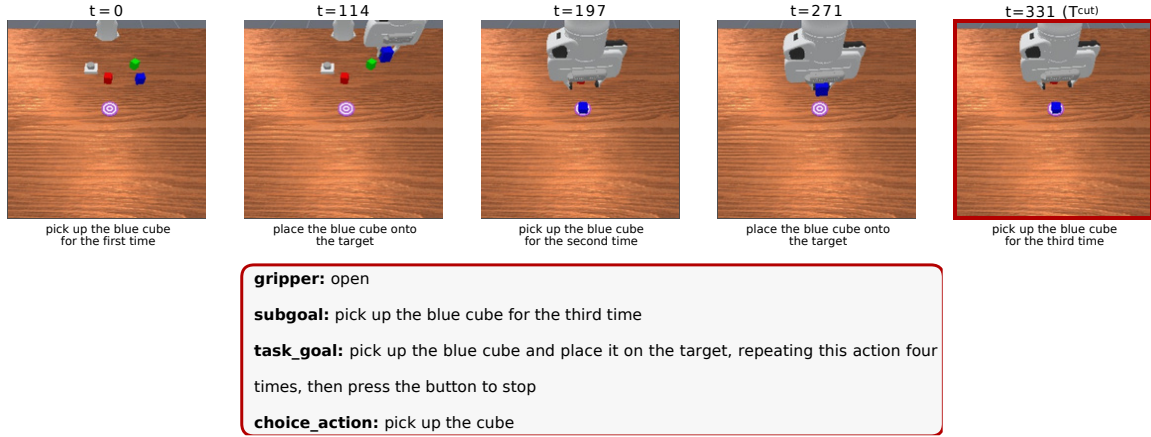


Fig. 3.3 **QA labels on an example episode.** Frames sampled from a single PickXtimes episode up to the cut frame T_{cut} (right), annotated with the per-timestep subgoal. The panel below lists the four QA labels extracted at T_{cut} , which pair with the NIR fitted to exactly these frames. Note that the final frame alone is compatible with multiple subgoal and task-goal instances, so distinguishing them requires information from earlier in the episode.

ordinal (PickXtimes: "... repeating this action two / three / five times"). This demands episode-specific memory at fine granularity.

3.3.3 NIR Flavor Configurations

Six flavors are fit per cut, each varying two independent factors: the NIR's initialization (meta-learned θ vs. independent random weights) and the supervisory signal used during fitting (pixel reconstruction, self-supervised DINO (Siméoni et al., 2025) features, CLIP features (Radford et al., 2021), or combinations). Holding one factor fixed while the other varies isolates whether gains come from meta-initialization, auxiliary semantic supervision, or both. Table 3.3 gives the full specification.

Table 3.3 **The Six NIR flavor configurations**, each fit following Section 3.2.4

Flavor	Init	Pixel	DINO (λ_{dino})	CLIP (λ_{clip})
1	random	✓	—	—
2	meta	✓	—	—
3	meta	—	✓(1.0)	—
4	meta	✓	✓(0.8)	—
5	random	✓	✓(0.8)	—
6	meta	✓	✓(0.4)	✓(0.4)

For flavors with auxiliary heads (3-6), the base NeRV architecture is modified by attaching a shared pooled feature (adaptive average pooling over the final feature map) feeding a linear DINO head, a linear CLIP head, or both. Target features are extracted once per episode by frozen encoders DINOv3 ViT-B/14 and CLIP ViT-L/14, each producing 768-dimensional features.

The fitting loss extends the reconstruction objective of Section 3.2.2:

$$\mathcal{L}_{\text{flavor}} = w_{\text{pixel}} \mathcal{L}_{\text{NeRV}} + w_{\text{dino}} \lambda_{\text{dino}} (1 - \cos(\hat{d}, d)) + w_{\text{clip}} \lambda_{\text{clip}} (1 - \cos(\hat{c}, c)), \quad (3.5)$$

where $\hat{d}$ and $\hat{c}$ are the NIR’s predicted DINO/CLIP features, d and c are the frozen encoders’ targets, and $w_{\text{pixel}}, w_{\text{dino}}, w_{\text{clip}} \in \{0, 1\}$ select active loss components according to the architecture flavor (see Table 3.3).

3.4 NIR Representation Results

3.4.1 Reconstruction Quality

Since the fitted NIRs are the stored representations this pipeline operates on, we check its quality by decoding directly from them. Figure 3.4 shows frames reconstructed from the stored weights of a single PickXtimes episode, with no access to the original video at decoding time. Decoded frames reach per-frame PSNR (Peak Signal-to-Noise Ratio) values between 27.0 and 32.1 dB at 64×64 resolution, with the lowest values on frames where the arm is in motion, consistent with the spectral bias of coordinate networks: high temporal frequencies are the hardest content to fit, and the reconstruction degrades through a visible motion blur. The arm, the tabletop, and object identities and positions nonetheless remain clearly identifiable at every timestep shown.

At corpus scale, fitting succeeds uniformly, as demonstrated in the full flavor example reconstructions of Figure 3.5. The dataset-wide view in Figure 3.6 shows every task and every flavor reconstructed at mean PSNR between roughly 27 and 37 dB, so the encoding stage produces a usable stored representation for all sixteen task types. Fidelity varies primarily with episode length, from roughly 36 dB on short episodes such as VideoUnmask (217 average steps, Table 3.1) to 27-28 dB on VideoPlaceOrder (1,134 average steps), which highlights a capacity-length tradeoff in NIR fitting.

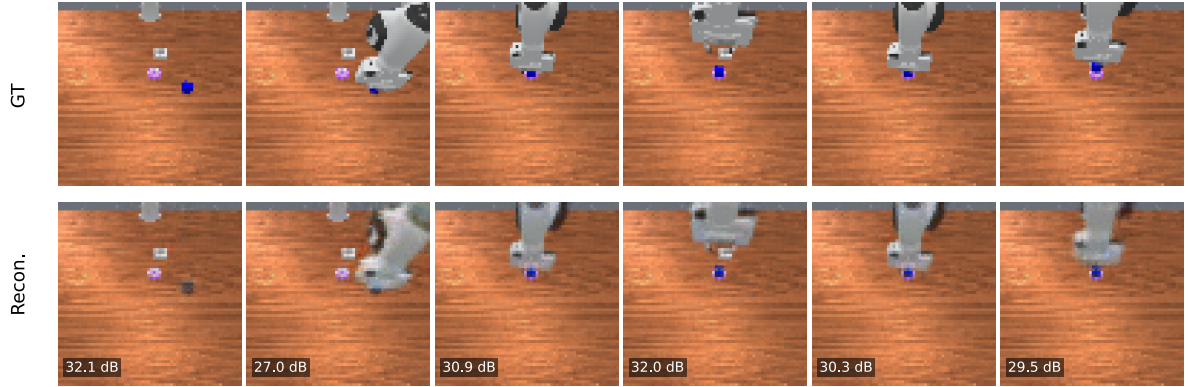


Fig. 3.4 **Qualitative frame fitting quality.** Ground truth (GT) frames (top) and decoded outputs from the fitted NeRV instance (bottom) across selected timesteps of a sample episode of task PickXtimes. Per-frame PSNR values (dB) are annotated in the bottom-left corner.

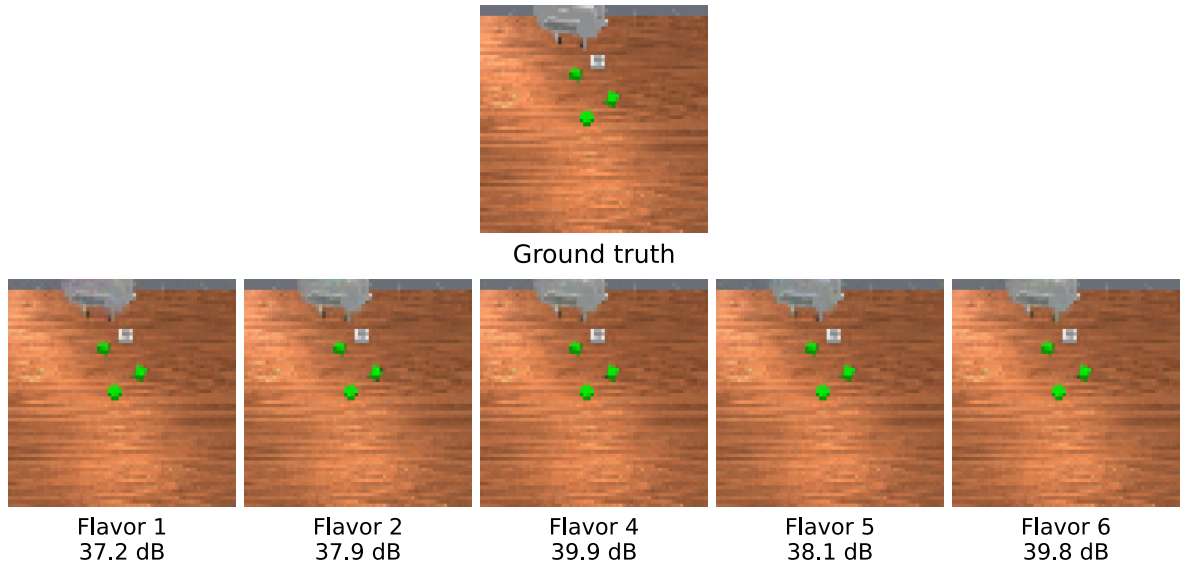


Fig. 3.5 **Post-fitting frame reconstruction across architecture flavors.** A single VideoRepick frame decoded from NIRs fitted under Flavors 1-6, with PSNR (dB) relative to the ground truth frame (top).

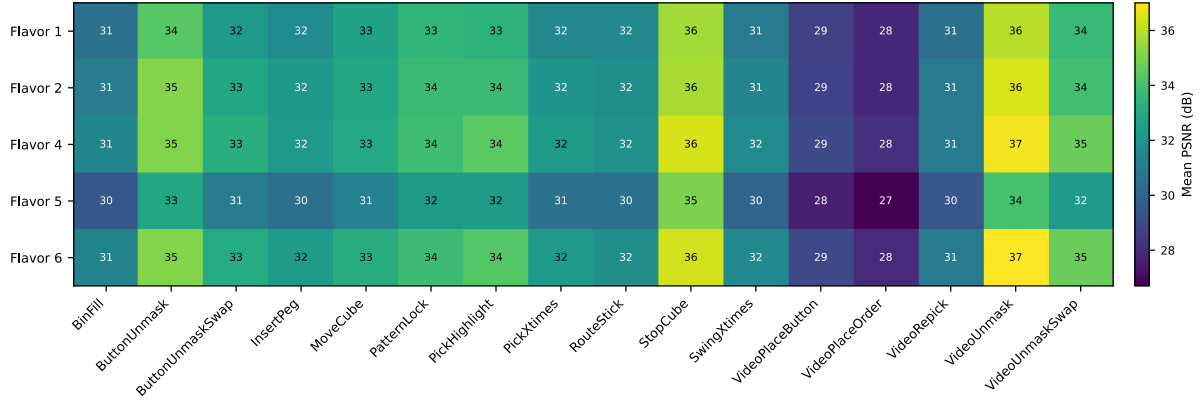


Fig. 3.6 **Dataset-wide NIR fitting performance.** Heatmap of mean PSNR (dB) achieved after fitting NIR instances, by task (x-axis) and flavor (y-axis).

3.4.2 Meta-Learned Weight Space

t-SNE embeddings were computed to assess whether the NIR weights show clear task-suite structure, given their initializations (van der Maaten & Hinton, 2008). No suite or task label was used in fitting the NIRs or computing the embedding, so any grouping reflects structure already present in the weight deltas.

Observing the t-SNE of meta-initialized weights of Flavor 2 (Table 3.3) on the left hand side of Figure 3.7, the Permanence and Imitation task episode weights form tight and well-separated clusters, while the Counting and Reference weights interleave with one another but are distinct from the other two suites. One qualification and one consequence follow from the meta-training setup. Since θ was trained on five tasks (Section 3.2.3), the weight space is label-free but not naive of task. Of the eight tasks embedded here, however, only three appeared during meta-training. Coherent clusters therefore form for tasks θ never adapted to, including the whole of the Reference and Imitation task suites shown, which is consistent with MAML’s premise that an initialization learned on a task subset supports adaptation to unseen ones (Finn et al., 2017).

The random-initialization on the right shows no visible separation by suite. All four categories are substantially more interspersed than in the meta-initialized representation, consistent with the claims that adaptation from the shared meta-learned initialization θ induces an organized common weight space among the NIRs.

Recoloring the same embedding of Flavor 2 by task, Figure 3.8, shows the organization is finer than memory category: VideoUnmask and ButtonUnmask tasks occupy adjacent but distinguishable subclusters within the Permanence cluster, as do MoveCube and InsertPeg

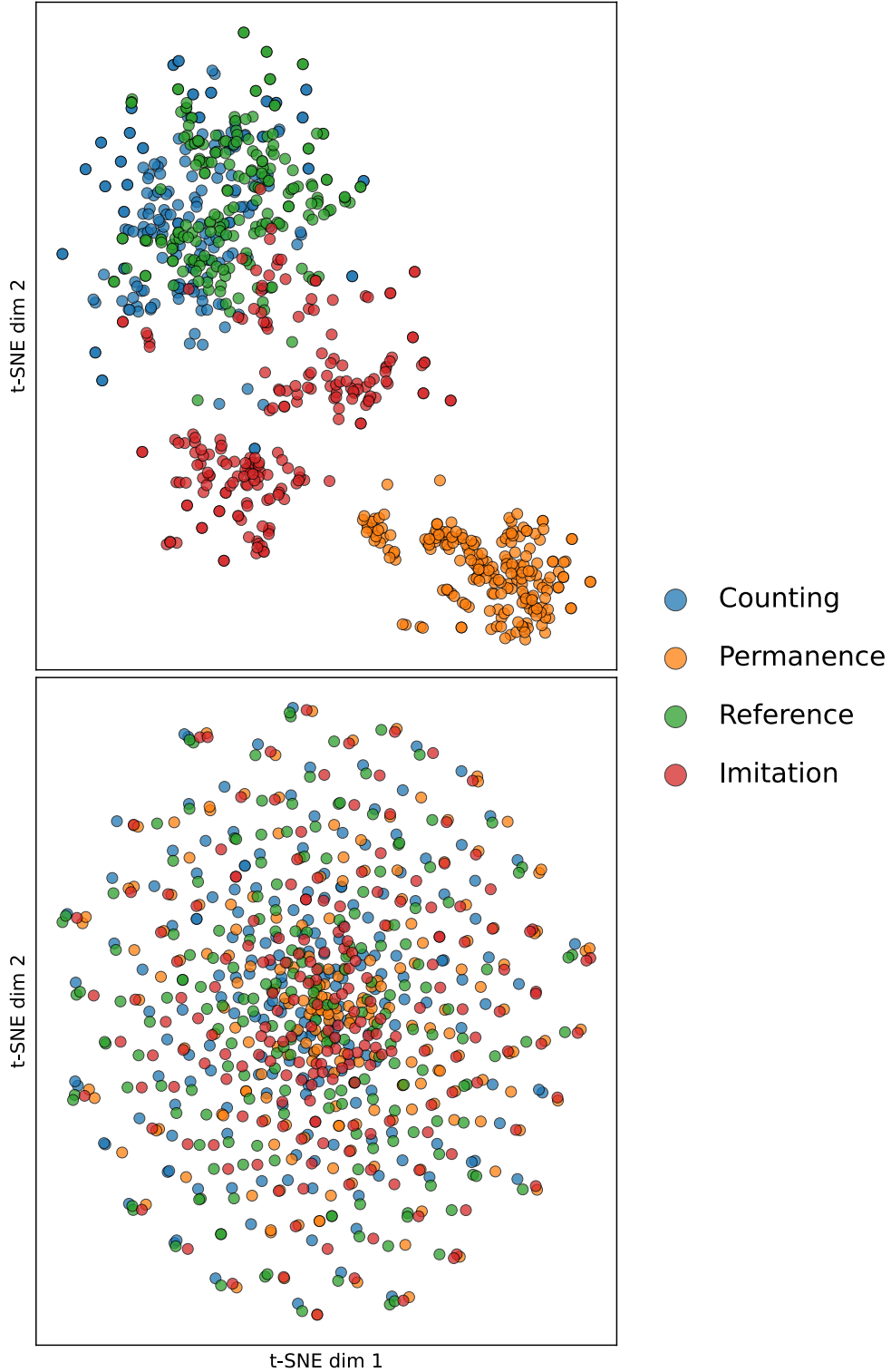


Fig. 3.7 **t-SNE of NIR weight space, meta (Flavor 2) versus random initialization (Flavor 1), colored by memory suite**, as defined in Table 3.1. Two independently fit t-SNE embeddings (PCA-100, perplexity 30) over a shared subset of 8 tasks (2 per suite: BinFill/PickXtimes, VideoUnmask/ButtonUnmask, PickHighlight/VideoRepick, MoveCube/InsertPeg, with 100 samples/task). Axes are in arbitrary t-SNE units.

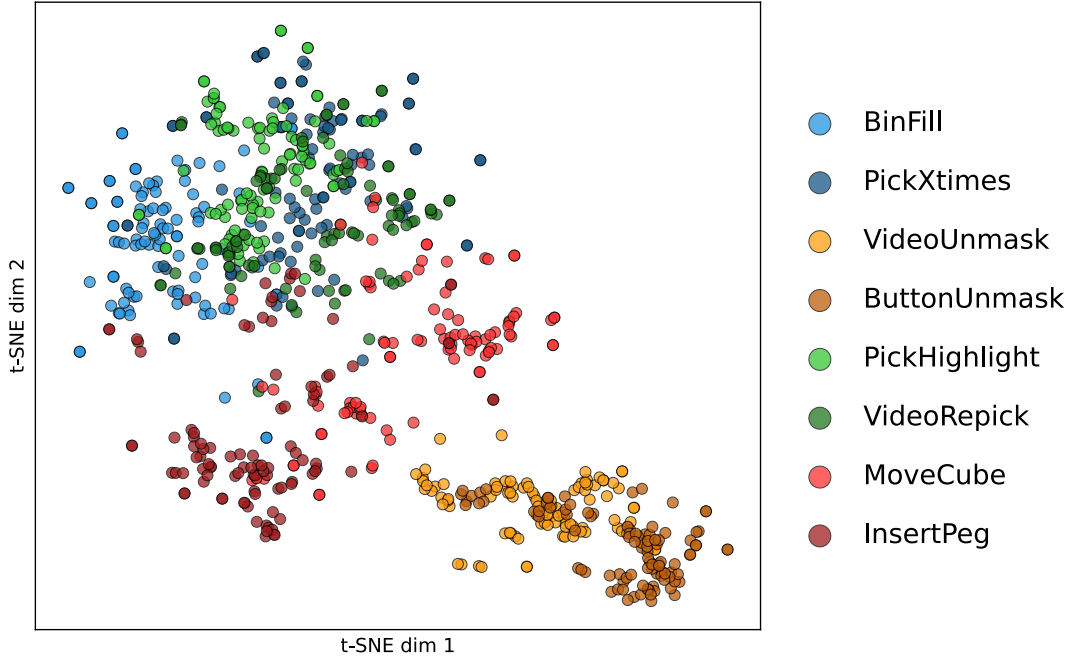


Fig. 3.8 **t-SNE of NIR weight space, organized by task within memory suite**, as defined in Table 3.1. The same meta-initialized (Flavor 2) embedding as Figure 3.7, recolored by task, with 100 samples per task. Tasks form adjacent but distinguishable sub-clusters within their suites.

within Imitation. The weight space is therefore reflecting what actually happens in the episodes, rather than only their broad memory category.

3.4.3 Linear Classification Probes

If task-relevant properties can be read from the weight deltas by a linear map, the information is present in the coordinates themselves rather than manufactured by a trained decoder. A linear probe was therefore fit directly on the weight deltas. Each episode’s delta is flattened into $\mathbf{x} = \text{vec}(\Delta) \in \mathbb{R}^D$ ($D \approx 1.26\text{M}$), standardized, and reduced to $k = 100$ principal components fit on train. A multinomial logistic regression,

$$p(y = c \mid \mathbf{z}) = \text{softmax}(\mathbf{W}\mathbf{z} + \mathbf{b})_c,$$

is trained on the projected features $\mathbf{z} \in \mathbb{R}^{100}$ with ℓ_2 regularization selected on a held-out split.

The results show that meta-initialized, pixel-supervised configurations provide task-relevant information to a linear probe at several multiples of chance, with Flavor 2 being the best performer. Table 3.4 shows Flavor 2 leading on three of the four question types and overall: it decodes `choice_action` and `subgoal` at roughly $3.7\times$ chance and `task_goal` at $3\times$. It also

gives gripper a gain of +10.7 percentage points over its majority class. Flavors 4 and 6 trail by a few points on most types, with Flavor 6 taking the best gripper performance at +13.1 points.

Table 3.4 **Linear Classification Probe on Raw NIR Weights.** PCA (100 components) followed by a linear classifier. Overall (%) pools correct predictions across all four question types (weighted by each type’s n). Sample sizes (train/eval): choice_action 923/909, gripper 1000/1000, subgoal 1000/1000, task_goal 705/694. Bold marks the best value per column.

Flavor	Init.	Heads			Probe Accuracy by Question Type (%)				Overall (%)
		Pixel	DINO	CLIP	choice_action	gripper	subgoal	task_goal	
<i>Chance Baseline</i>	–	–	–	–	<i>13.1</i>	55.5	8.0	4.8	–
1	Random	✓	–	–	14.7	55.5	5.2	3.7	21.3
2	Meta	✓	–	–	48.3	66.2	29.9	14.3	41.6
3	Meta	–	✓	–	7.6	55.5	5.1	3.0	19.3
4	Meta	✓	✓	–	48.1	67.5	25.9	12.8	40.5
5	Random	✓	✓	–	13.6	55.5	5.6	4.2	21.2
6	Meta	✓	✓	✓	43.8	68.6	28.6	10.8	40.1

Neither meta-initialization nor pixel supervision suffices alone. Randomly initialized Flavors 1 and 5 sit at, and some below, baseline on every question type despite supervision identical to Flavors 2 and 4. Additionally, Flavor 3 (meta-initialized but fit on DINO features without pixel reconstruction) collapses below baseline on three of four types. The gripper column further proves this, as every random or pixel-free configuration scores exactly the 55.5% majority rate, while under a meta-learned origin the same label becomes decodable at 66 to 69%.

Reconstruction fidelity also does not explain these differences. Averaging Figure 3.6’s per-task PSNR across all sixteen task types yields a spread of under 2 dB between the five pixel-supervised flavors, while probe gain over baseline ranges from effectively zero to nearly $4\times$ chance across the same configurations. A model can reconstruct an episode’s frames equally well whether or not the resulting weights are linearly readable, so the pixel loss is doing representational work that pixel fidelity does not measure, and only from a shared origin does that structure land in coordinates a linear map can use (Section 2.3).

These probes give the next chapter its starting point. Flavor 2 is carried forward as the primary configuration for its highest performance and simplest configuration, though Flavors 4 and 6 remain close enough that the full set of six flavors are re-examined. Whether a language model can exceed what a linear map already recovers, and whether it can extract anything from the randomly initialized weights the probe struggles to read, are the questions the next chapter takes up.

Chapter 4

Projecting NIR Weights into Language Space

Chapter 3 compressed each episode into a weight delta $\Delta = \phi_{\text{fit}} - \theta$, a compact summary of everything the NIR observed, expressed as a change from a shared meta-initialization. With that dataset of fitted deltas in place, the probes identified Flavor 2, meta-initialized with pixel reconstruction alone, as the best-performing configuration, decoding task-relevant labels at up to $3.7\times$ the chance baseline. It is therefore the primary configuration in this chapter, with all six flavors re-examined here. This chapter addresses the remaining part of the problem formulation (Section 3.1): how Δ is turned into something a language model can read, how the resulting system is trained, and how it is evaluated.

Turning the weights into a representation the language model can use involves three steps (see Figure 4.1):

1. it is converted into a sequence of token embeddings in the same space as the language model’s own token embeddings (Section 4.1.1);
2. those tokens are inserted into a prompt alongside the natural-language question (Section 4.1.2); and
3. the whole system, consisting of a projector and the language model itself, is trained and fine-tuned to make use of the injected tokens (Sections 4.1.3).

The language model used throughout is TinyLlama-1.1B-Chat (Zhang et al., 2024), chosen for its small size relative to other openly available chat models, which keeps training and the two-stage fine-tuning experiments of Section 4.1.3 computationally tractable.

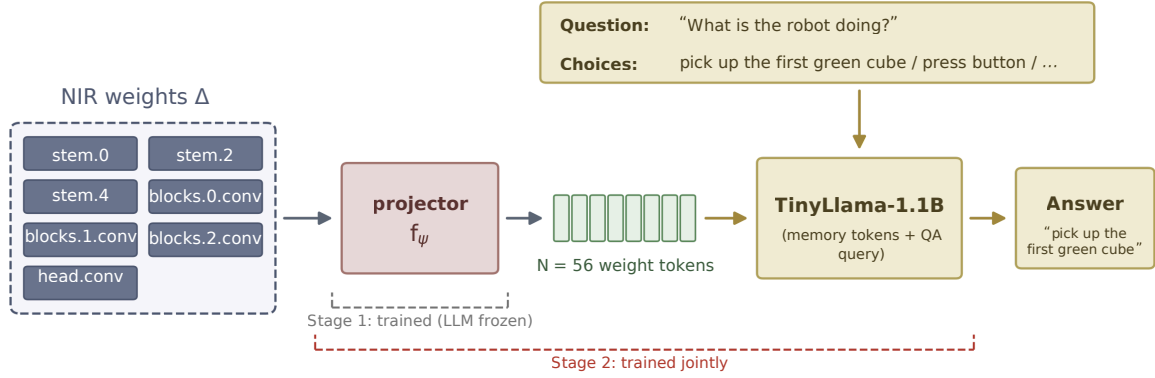


Fig. 4.1 **Chapter Overview.** Schematic of the projection pipeline: a weight delta Δ (one tensor per NIR layer) is projected tensor-by-tensor into a shared latent width, pooled into a fixed-length token sequence, and spliced into a frozen then fine-tuned language model alongside a natural-language question. The model outputs an answer without ever seeing the original video frames.

4.1 Methodology

4.1.1 From Weights to Token Sequences

Since Δ is a collection of tensors, one per NIR weight matrix, flattening them into one vector through a single linear layer would discard the layer-wise structure that gives the weights their meaning, and would likely not scale well as the NIR architecture changes. Instead, following the INR2ARRAY approach established by Zhou et al. (2023), each tensor is projected independently, and the results assembled into a single token sequence (Table 4.1).

Table 4.1 **The tensor-shape pipeline** from a raw weights to language-model tokens.

Step	Input shape	Output shape
1. Weight delta (one tensor)	—	$W \in \mathbb{R}^{o \times i}$ (or $o \times i \times k_h \times k_w$)
2. Fold to rows	$\mathbb{R}^{o \times i \times k_h \times k_w}$	$W' \in \mathbb{R}^{o \times (i \cdot k_h \cdot k_w)}$
3. Per-tensor projection	$\mathbb{R}^{o \times (\cdot)}$	$X \in \mathbb{R}^{o \times \tilde{c}}$
4. Pooling	$\mathbb{R}^{o \times \tilde{c}}$	$X \in \mathbb{R}^{t_{pl} \times \tilde{c}}$
5. Concatenate across tensors	n groups of $\mathbb{R}^{t_{pl} \times \tilde{c}}$	$\mathbb{R}^{(n \cdot t_{pl}) \times \tilde{c}}$
6. Map to hidden size	$\mathbb{R}^{(n \cdot t_{pl}) \times \tilde{c}}$	$\mathbb{R}^{(n \cdot t_{pl}) \times d_{\text{hidden}}}$

Per-Tensor Projection

Each weight tensor is treated as a set of row vectors, one row per output unit (linear layer) or per output filter (for a convolutional layer, after folding its input and spatial dimensions into a single row). A dedicated linear map is learned for each tensor, projecting every row into a shared latent width $\tilde{c} = 256$:

$$\text{PROJ}_i : \mathbb{R}^{\text{row width of tensor } i} \rightarrow \mathbb{R}^{\tilde{c}}. \quad (4.1)$$

The projector applies PROJ_i across $n = 7$ active weight tensors (the linear stem layers and convolutional upsampling blocks of the NIR (Section 3.2.2)). Because a different PROJ_i is learned for every tensor, tensors of different shapes and roles (e.g. an early stem layer versus a later upsampling block) are not forced through the same projection, and yet all land in the same shared latent space.

Pooling and Sequence Assembly

A tensor with o output rows produces o tokens after projection, and o varies considerably across layers. To keep the resulting sequence a fixed, predictable length regardless of the underlying NIR architecture, every tensor’s tokens are pooled down to a fixed number per layer, t_{pl} , via adaptive average pooling: the o tokens are grouped into $t_{\text{pl}} = 8$ contiguous chunks, with each chunk averaged (the last chunk is padded by repetition when o does not divide evenly).

The pooled token groups from all n tensors are concatenated into one sequence of $n \cdot t_{\text{pl}}$ tokens, still in the shared latent width $\tilde{c}$. A final shared projection,

$$\text{to_hidden}(\cdot) = \text{Linear}(\text{LayerNorm}(\cdot)) : \mathbb{R}^{\tilde{c}} \rightarrow \mathbb{R}^{d_{\text{hidden}}}, \quad (4.2)$$

maps every token into the language model’s hidden dimension d_{hidden} , producing the weight-token sequence spliced into the model’s input. Together, the per-tensor projections and this shared final layer make up the trainable projector f_{ψ} defined in Section 3.1.

4.1.2 Integrating with the Language Model

Prompt Construction

Each item’s question, candidate choices, and target answer are constructed once from the episode’s stored metadata (Table 3.2) and the result serialized into a two-turn conversation used at both training and evaluation time. Each training example pairs one episode’s weight delta with a natural language question. The question text contains a placeholder, `<point>`, expanded

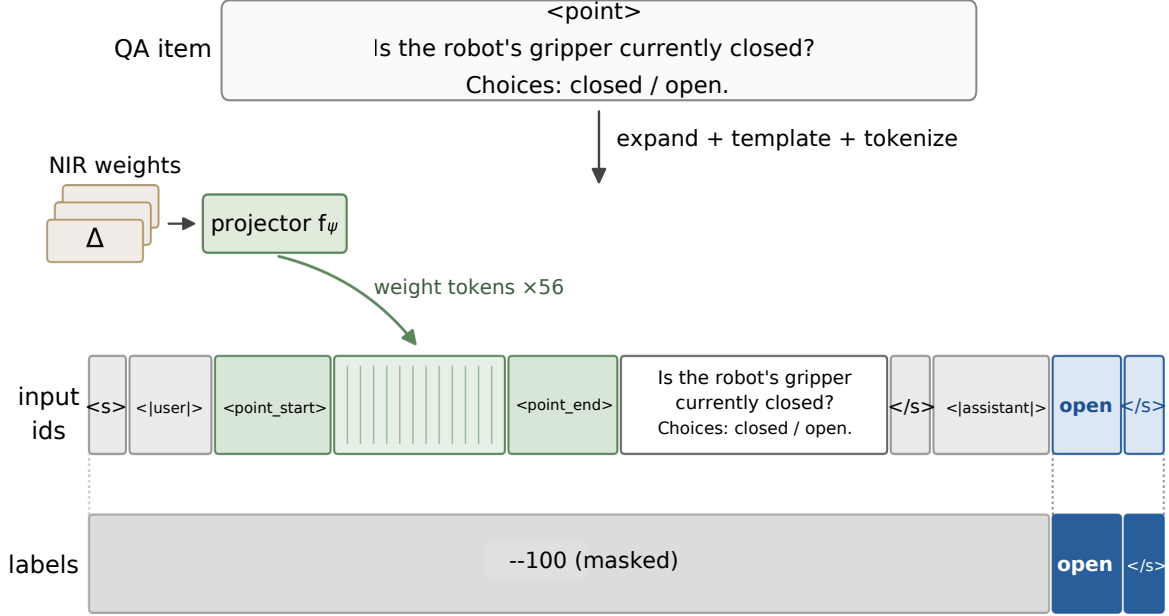


Fig. 4.2 **Prompt Construction.** Shows the substitution of a single `<point>` placeholder into an explicit `<point_start>`, $N \times$ `<point_patch>`, `<point_end>` span, the chat-template formatting of the resulting human turn and target answer, and the loss mask applied to answer tokens only.

before tokenization into a fixed-length span:

$$\langle \text{point_start} \rangle \langle \text{point_patch} \rangle \times N \langle \text{point_end} \rangle,$$

where $N = n \cdot t_{\text{pl}}$ matches the projector's sequence length, and `<point_start>`, `<point_end>` are special vocabulary tokens framing the span. Before the forward pass, the projector's output embeddings replace the $N \times$ `<point_patch>` placeholder embeddings, while standard text tokens keep their pretrained embeddings.

Each question presents up to four candidate answer choices in the prompt text under a fixed template ("*Choices: [choice 1] / [choice 2] / ...*"), read from a per-item, per-question-type candidate set. The full example is formatted as a two-turn conversation in TinyLlama-Chat's native template (Zhang et al., 2024), with cross-entropy loss computed only on the assistant's response tokens. The prompt, choice, and weight tokens are masked out, following the input construction of LLaNA (Amaduzzi et al., 2024). Figure 4.2 illustrates this full process on a sample example.

4.1.3 Training the Projector

Training Objective

The projector is trained via teacher-forced language modeling: weight-tokens, question, and target answer pass through the model in one forward pass under causal masking, with cross-entropy computed only over the answer tokens (Section 4.1.2). Each example pairs one episode with one question of one of the four types discussed in Section 3.3.2. Although candidate choices appear in the prompt, the model is trained to reproduce the answer as free-form text rather than an index into the choice list.

Two-Stage Training

Our approach to projecting a non-linguistic representation into a language model’s embedding space via a trainable projector closely follows the training of LLaNA (2024) and LLaVA (H. Liu et al., 2023):

In **Stage 1**, the language model is frozen, and only the projector and the new placeholder-token embeddings are updated.

In **Stage 2**, training resumes with the language model unfrozen, and the projector and full language model are fine-tuned jointly at a substantially lower learning rate to limit catastrophic forgetting of general language ability. This tests whether a language model that never saw weight-tokens during pretraining can adapt to this new input form.

Contrastive Weight-Token Swap Loss

Training on answer-generation alone does not guarantee the model is actually using its weight-tokens: a projector could learn to produce tokens the language model largely ignores, predicting answers from the question text alone. To penalize this, each training example is evaluated twice: once with its own weight-tokens, producing loss $\mathcal{L}_{\text{real}}$, and once with another randomly chosen example’s weight-tokens substituted within the same batch, with the question and answer left unchanged, producing $\mathcal{L}_{\text{swap}}$. A hinge penalty,

$$\mathcal{L}_{\text{contrastive}} = \max(0, m + \mathcal{L}_{\text{real}} - \mathcal{L}_{\text{swap}}), \quad (4.3)$$

with margin $m = 0.5$, is added to give total loss $\mathcal{L} = \mathcal{L}_{\text{real}} + \lambda_c \cdot \mathcal{L}_{\text{contrastive}}$. A model that ignores the weight-tokens entirely would produce $\mathcal{L}_{\text{real}} \approx \mathcal{L}_{\text{swap}}$, incurring a penalty close to the margin regardless of which episode’s tokens are attached to a given question. The hinge can only shrink if the model’s loss genuinely depends on receiving the correct weight-tokens.

Because answer lengths vary sharply across question types, both $\mathcal{L}_{\text{real}}$ and $\mathcal{L}_{\text{swap}}$ are averaged within each example first, and only then averaged across the batch, so no question type dominates the gradient simply by contributing more tokens.

4.1.4 Evaluation via Choice-Scoring

For each question, every candidate answer choice is scored under the trained model as a negative log-likelihood, and the choice with the lowest score is taken as the model’s prediction.

Prior Debiasing

A language model carries its own priors over which candidate strings are more fluent or common in its pretraining data, independent of the weight-tokens. To isolate the weight-tokens’ effect, each candidate is scored twice, with and without weight-tokens spliced in. The reported score is then the difference:

$$\text{score}(c) = \text{NLL}(c \mid \text{prompt} + \text{weight-tokens}) - \text{NLL}(c \mid \text{prompt only}). \quad (4.4)$$

This debiased score reflects how much the weight-tokens shift the model’s preference for an answer, rather than the model’s absolute preference for it.

4.1.5 End-to-End Inference

Each episode’s delta $\Delta = \phi_{\text{fit}} - \theta$ is obtained offline by the fitting procedure of Section 3.2.4. To answer a question about an episode, Δ passes through the trained projector f_{ψ} , and the resulting weight-tokens are inserted into the placeholder span of a prompt containing the natural-language question (Section 4.1.2). The language model then scores each candidate choice (Section 4.1.4). An example is illustrated in Figure 4.3.

4.2 Experiment Design

4.2.1 Evaluation Protocol

To ensure a balanced evaluation and prevent shortcut learning in multiple-choice question answering, we apply two dataset construction constraints:

Per-Task Balanced Sampling. To prevent task imbalance when sampling from a fixed item budget N_{budget} (the total number of items drawn for a given split, with 4,000 for train and

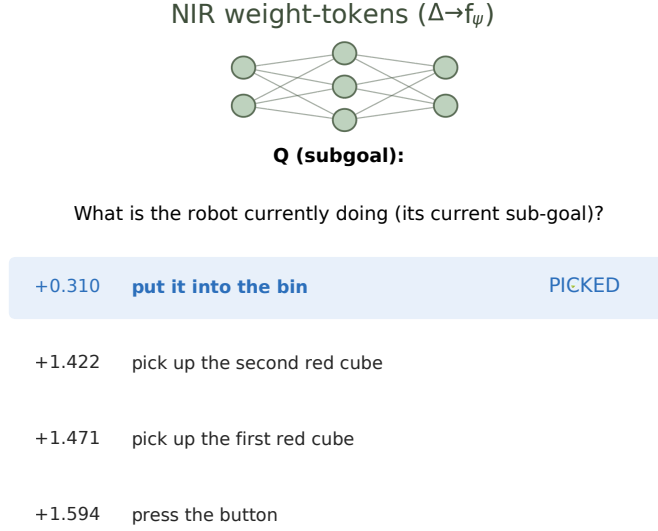


Fig. 4.3 **Choice-scoring at inference.** A test item’s candidate answers are each scored by their prior-debiased negative log-likelihood (Eq. 4.4); values are shown beside each candidate, and the prediction is the minimum-scoring choice (highlighted).

800 each for val and test), each of the $N_{\text{tasks}} = 16$ tasks’ contribution is capped independently:

$$N_{\text{cap}} = \max \left(1, \left\lfloor \frac{N_{\text{budget}}}{N_{\text{tasks}}} \right\rfloor \right) \quad (4.5)$$

giving $N_{\text{cap}} = 250$ items per task for train and 50 per task for validation and test, to avoid any particular task category from dominating either training or validation.

Intra-Task Distractor Pooling. For `task_goal` and `subgoal` queries, multiple-choice distractors are sampled exclusively from the ground-truth task’s label vocabulary rather than the global vocabulary. Pooling across tasks would let the model answer by recognizing coarse task identity alone; restricting to the same task forces reliance on what the weight-tokens encode about the specific goal or sub-action.

Because both constraints apply per item, the candidate count K_j is not fixed across question types. `gripper` is always binary ($K_j = 2$) and `subgoal` always presents four candidates. `choice_action` and `task_goal` are capped at four but a task’s native action field, or its `task_goal` vocabulary at a given point, can be too sparse to supply four distinct options. These items are retained with $K_j < 4$ where distinct options exist and excluded where a full, unambiguous set cannot be constructed. This leads to item counts differing across question types drawn from the same corpus: `gripper` and `subgoal` retain the full count, while `choice_action` and `task_goal` are smaller.

4.2.2 Metrics and Reporting

Accuracy. For each item, every candidate choice is scored with the debiased NLL of Section 4.1.4, and the prediction is the argmin. Accuracy for question type t is

$$\text{Acc}_t = \frac{1}{n_t} \sum_{j=1}^{n_t} \mathbb{1}[\hat{y}_j = y_j], \quad (4.6)$$

and overall accuracy pools items across all four types before averaging, a micro-average weighted by each type’s item count. All accuracies are reported as mean $\pm$ standard deviation across 2 training seeds, with data splits held fixed.

Chance baseline. Since each item presents its own explicit candidate set of size K_j , the baseline for question type t is the average per-item chance accuracy,

$$\frac{1}{n_t} \sum_{j=1}^{n_t} \frac{1}{K_j}. \quad (4.7)$$

For gripper ($K_j = 2$ throughout) this is exactly 50.0%, and for subgoal ($K_j = 4$ throughout) exactly 25.0%. For choice_action and task_goal, the items retained with $K_j < 4$ (Section 4.2.1) raise the average above 25%, since $1/K_j$ grows as K_j shrinks: 32.0% for choice_action and 29.5% for task_goal.

Cohen’s kappa. Gains against a chance baseline can still be inflated by a model that tracks the label distribution rather than the input, so chance-corrected agreement (Cohen, 1960) is reported alongside accuracy. Per question type, from the per-item predictions,

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \quad (4.8)$$

where p_o is observed accuracy and p_e the agreement expected by chance under the marginal distributions of true and predicted labels. A predictor that echoes the majority class receives $\kappa = 0$ regardless of raw accuracy, so κ confirms that gains reflect input-dependent prediction. Values are interpreted against the bands of Landis and Koch (1977).

Random-input control. Accuracy above a chance baseline establishes that the trained system outperforms guessing, but not that its performance depends on the episode encoded in the weight tokens. A fine-tuned model can exploit regularities in the question text, the structure of the candidate sets, and the label marginals absorbed during training, none of which involve memory. To measure the episode-specific contribution directly, every configuration is therefore re-evaluated with the weight delta Δ replaced by Gaussian noise of matching shape, drawn independently per tensor and passed through the trained projector, the other aspects of

evaluation unchanged. The substitution removes only the information that derives from the episode itself, therefore any margin a configuration retains over its noise-substituted counterpart is attributable to episode content alone.

4.2.3 Projector Training Configuration

Optimization follows the two-stage scheme of Section 4.1.3: Stage 1 aligns the projector with a frozen language model, and Stage 2 fine-tunes end-to-end at a lower learning rate with cosine decay and 3% warmup. Table 4.2 summarizes the configurations for both stages.

Table 4.2 **Hyperparameter settings** and optimization configuration for Stage 1 (projector alignment) and Stage 2 (end-to-end fine-tuning).

Setting	Stage 1	Stage 2
Language Model	Frozen	Unfrozen (Stage 1 init)
Trainable Parameters	Projector + Token Embeddings	Projector + Full LLM
Learning Rate	1×10^{-4}	2×10^{-5} (Cosine, 3% Warmup)
Epochs	15	3
Batch Size	4	1 (Grad. Accum. $\times 8$)
Contrastive Loss (λ_c / m)	1.0 / 0.5	1.0 / 0.5
Projector Dim. ($\tilde{c}$)	256	256
Tokens per Layer (t_{pl})	8	8
Active Tensors (n)	7	7
Total Weight-Tokens (N)	56	56

4.3 Main Results

4.3.1 Question-Answering Results

The central result of this chapter is that a language model, given 56 projected weight-tokens and no access to any frame, exceeds its chance baseline on every question type after Stage 2 (Table 4.3): +16.1 percentage points on `choice_action`, +9.5 points on `subgoal`, +19.1 points on `gripper`, and +3.5 points on `task_goal`. Flavor 2 (meta-learned initialization, pixel-only supervision), provides the strongest performance, and is reported here. Figure 4.4 shows this mechanism on example items of a `subgoal` question answered correctly against a `task_goal` question answered incorrectly.

With the language model frozen, Stage 1 clears chance substantially only on `gripper` (+12.4pp), with the other three question types sitting at their floors. A frozen language model, given only a trained projector, can therefore read instantaneous state out of the weight-tokens. Unfreezing changes this, as fine-tuning multiplies the `choice_action` gain roughly $8\times$ and

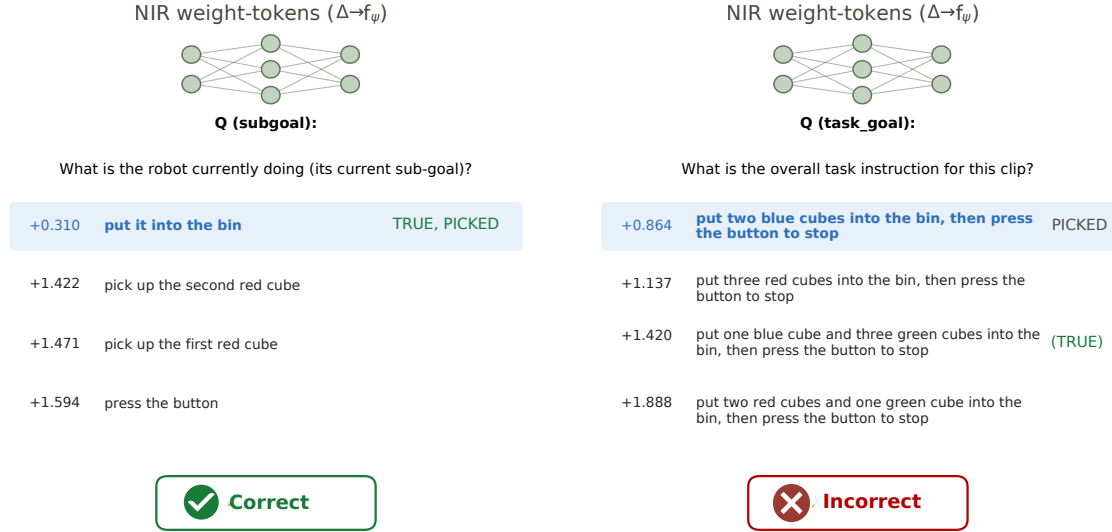


Fig. 4.4 **Choice-scoring at inference**, illustrated on two samples (Flavor 2). Every candidate answer is scored by its prior-debiased negative log-likelihood (Eq. 4.4). Scores are shown beside each candidate and the prediction is the minimum. Left: a subgoal item answered correctly, with the true candidate separated from the distractors by a clear margin. Right: a task_goal item in which all candidates are near-duplicate goal instances of the same task and the debiased scores fail to isolate the true instance.

Table 4.3 **Main End-to-End QA Performance (Flavor 2: Meta-Learned + Pixel Head)**. Debiased accuracy (mean $\pm$ std across 2 seeds) after subtracting the LLM prior NLL. Chance is the per-item random-guess baseline. The OVERALL row uses the same item-weighted pooling as OVERALL accuracy. Gain is Stage 2 accuracy minus chance, in percentage points. Stage 2 fine-tuning yields substantial gains on choice_action, gripper, and subgoal. Cohen’s κ is reported for Stage 2.

Question Type	Chance (%)	Stage 1 (%)	Stage 2 (%)	Gain (pp)	Stage 2 κ
choice_action	32.0	33.9 ± 0.6	48.1 ± 2.8	+16.1	0.444
gripper	50.0	62.4 ± 8.2	69.1 ± 4.5	+19.1	0.371
subgoal	25.0	29.2 ± 0.9	34.5 ± 2.7	+9.5	0.326
task_goal	29.5	29.1 ± 0.6	33.0 ± 0.7	+3.5	0.309
OVERALL	34.6	39.6 ± 1.8	47.2 ± 2.6	+12.6	0.446

doubles subgoal’s. Progress information is present in the tokens, but consuming it requires reshaping the language model itself, consistent with LLaNA’s findings (Amaduzzi et al., 2024).

`task_goal` is the most demanding label in the study and carries the narrowest margin, +3.5 pp with $\kappa = 0.309$. Under the intra-task distractor pooling of Section 4.2.1, every candidate is a near-duplicate goal instance from the same task, differing in a single attribute such as a color or a repetition count, so only instance-level episode content can separate them. That a margin persists under this construction indicates the weight-tokens carry goal-relevant information.

Across question types, Cohen’s κ of 0.31 to 0.44 indicates fair to moderate chance-corrected agreement, and the pooled Stage 2 value of 0.446 approaches the moderate band of Landis and Koch (1977), meaning the gains reflect input-dependent prediction rather than merely tracking the label distribution.

4.3.2 Flavor Ablation Results

The ablation asks which of the two design factors, initialization and fitting supervision, the end-to-end result depends on. Initialization is the clearer of the two. Flavor 2 exceeds its random-origin counterpart Flavor 1 by +9.2 percentage points overall, with the widest separation on gripper, where random-origin configurations sit at their 50% floor and meta-initialized ones reach 66 to 69%. Cohen’s κ moves in step (0.446 against 0.347), so the gap is not an artifact of label marginals.

Table 4.4 Systematic Ablation of NIR Configurations (Stage 2 Debiased Accuracy, Test Split). Accuracy is mean $\pm$ std across 2 seeds. Meta-learned initializations (θ) outperform random origins across configurations; auxiliary DINO/CLIP supervision yields no overall gain over pixel-only reconstruction, with Flavor 6 showing a selective advantage on subgoal and `task_goal` (Section 4.3.2). Cohen’s κ (Landis and Koch, 1977) is reported as an additional chance-corrected agreement measure. Δ_{noise} is the overall accuracy retained over the matched noise control of Section 4.2.2 (control=none minus control=random; mean $\pm$ std of per-seed differences, single seed for Flavor 5): meta-initialized configurations with a pixel head lose 8 to 11 points under the substitution, while the remaining configurations are near unchanged, so only the former’s margins are episodic in origin.

Flavor	Init.	Heads			Accuracy by Question Type (%)				Overall (%)	Δ_{noise} (pp)	κ
		Pixel	DINO	CLIP	action	gripper	subgoal	task_goal			
<i>Chance Baseline</i>	–	–	–	–	32.0	50.0	25.0	29.5	–	–	–
1	Random	✓	–	–	41.5 $\pm$ 0.7	50.2 $\pm$ 0.5	28.6 $\pm$ 1.3	29.4 $\pm$ 0.4	38.0 $\pm$ 0.5	0.4 $\pm$ 0.1	0.347
2	Meta	✓	–	–	48.1 $\pm$ 2.8	69.1 $\pm$ 4.5	34.5 $\pm$ 2.7	33.0 $\pm$ 0.7	47.2 $\pm$ 2.6	11.4 $\pm$ 0.8	0.446
3	Meta	–	✓	–	37.6 $\pm$ 0.6	52.5 $\pm$ 1.8	25.3 $\pm$ 1.0	30.2 $\pm$ 1.9	37.0 $\pm$ 0.8	2.5 $\pm$ 0.8	0.338
4	Meta	✓	✓	–	46.1 $\pm$ 4.7	65.6 $\pm$ 0.9	31.6 $\pm$ 2.0	32.8 $\pm$ 0.6	45.0 $\pm$ 2.1	9.7 $\pm$ 1.8	0.421
5	Random	✓	✓	–	40.6 $\pm$ 0.8	51.6 $\pm$ 1.2	27.4 $\pm$ 1.1	31.5 $\pm$ 2.1	38.3 $\pm$ 0.9	1.9 $\pm$ 0.1	0.351
6	Meta	✓	✓	✓	44.7 $\pm$ 3.1	66.2 $\pm$ 1.8	36.2 $\pm$ 2.6	33.4 $\pm$ 0.1	46.1 $\pm$ 0.5	8.4 $\pm$ 1.8	0.434

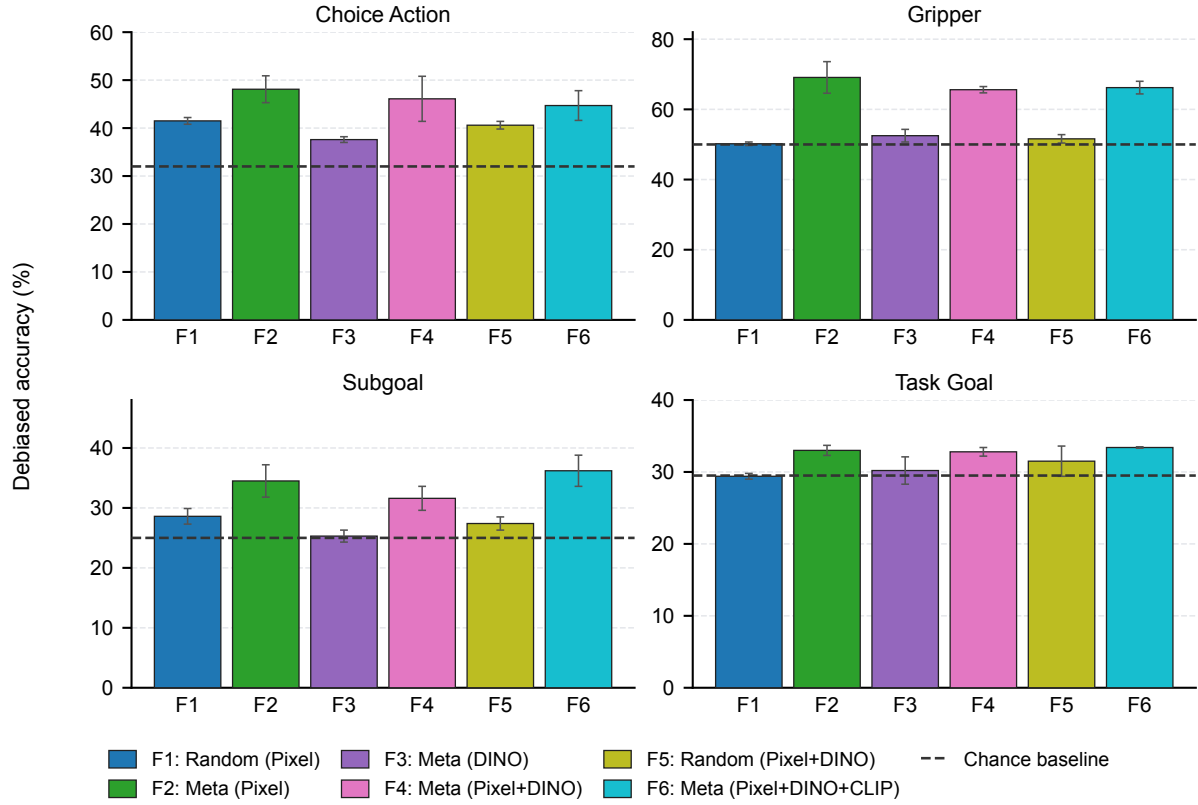


Fig. 4.5 Flavor Ablation by Question Type (Stage 2 Debiased Accuracy). Debiased accuracy for each NIR flavor configuration (Table 4.4), shown separately per question type. Dashed lines mark each type’s chance baseline. Bars are the mean across 2 seeds, with whiskers denoting ± 1 standard deviation, and axis ranges differing per panel. Meta-initialized flavors with a pixel head (2, 4, 6) clear the baseline decisively on choice_action, gripper, and subgoal, with the sharpest initialization separation on gripper, where random origins sit at or below the reference. Margins on task_goal are small for every configuration, with only the meta-initialized flavors exceeding the baseline.

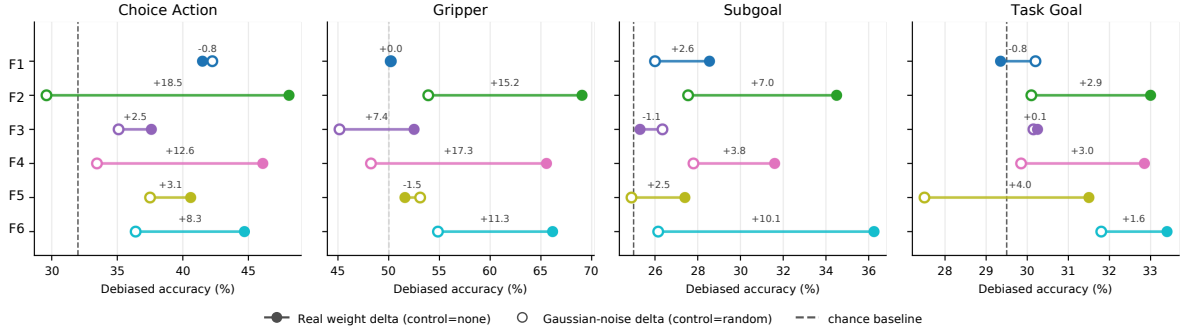


Fig. 4.6 Matched noise control by flavor and question type (Stage 2 Debiased Accuracy, Test Split). Debiased accuracy with real weight deltas (filled markers) against the same checkpoints evaluated with the delta replaced by Gaussian noise passed through the trained projector (open markers), for each NIR flavor of Table 4.4. Connector length is the episode-specific contribution, annotated in points. Dashed lines mark each type’s per-item chance baseline. Meta-initialized flavors with a pixel head (2, 4, 6) fall to or below chance under the substitution, Flavor 1 sits almost exactly on top of itself across both markers on every panel, and Flavor 3 shifts only modestly, so only the former group’s margins depend on episode content.

The effects of supervision choice are more modest and run largely in one direction. Flavor 3, meta-initialized but fitted to DINO features alone, is the weakest configuration in the table at 37.0%, below even random-origin Flavor 1. A shared origin without reconstruction is therefore worth less than reconstruction without a shared origin: whatever the auxiliary head contributes, removing the pixel objective is a net loss. The remaining DINO comparisons agree. Flavor 4 trails its DINO-free counterpart Flavor 2 by +2.2 percentage points overall, and Flavor 5 tracks Flavor 1 within a point on every question type, leaving initialization as the operative factor at this scale, rather than the auxiliary head.

Flavor 6 is the one configuration mostly dependent on the question type. It trails Flavor 2 overall and loses ground on choice_action and gripper, yet it posts the best subgoal and task_goal values in the table, an ordering also visible in Figure 4.5. These are the two question types whose answers require more fine-detail natural-language descriptions such as cube color and number, which is at least suggestive given that CLIP’s image embedding is trained against language. This is the only place in the ablation where feature supervision improves performance, and it does so where language alignment would be expected to matter.

The noise control separates the meta-initialized and random-origin configurations more sharply than accuracy alone (Figure 4.6; Table 4.4, Δ_{noise} column). For Flavor 2, replacing the delta with noise removes most of the measured performance: overall accuracy falls from 47.2% to 35.9%, choice_action loses 18.5 points and lands below its 32.0% chance baseline, gripper falls by 15.2 points to 53.9%, and subgoal by 7.0 points to 27.6%, leaving every

question type at or near its per-item chance level. The below-chance `choice_action` value indicates that the fine-tuned model conditions on the token span rather than ignoring it, producing wrong answers when the tokens are uninformative.

The smallest separation for Flavor 2 with this control is on `task_goal` (+2.9 points), consistent with the narrow instance-level margin reported above. Flavors 4 and 6 behave like Flavor 2, losing 9.7 and 8.4 points overall under the same substitution. Flavor 1 does not move: its overall accuracy is 38.0% with real deltas and 37.7% with noise, and no question type shifts by more than 2.6 points. The apparent +9.5-point margin on `choice_action` therefore survives the removal of all episode information, and must derive from question and candidate statistics learned during fine-tuning rather than from anything the random-origin weights encode. Flavors 3 and 5 likewise move by only 2.5 and 1.9 points overall. What the trained model recovers from configurations the linear probe reads as empty is consequently not episode content, and the shared origin is what makes these weight tokens informative about the episode content at all.

The suite decomposition of Table 4.5 addresses whether the pooled result is broad or carried by one memory type. It is broad. Per-suite accuracy spans a narrow band for both flavors, roughly 44 to 52%, so temporal, spatial, object, and procedural memory all contribute, and the overall figure is not the product of a single tractable suite. Within that band, two patterns stand out. Reference tasks are the weakest suite for both flavors, and it is also the suite with the longest episodes (Table 3.1). Permanence tasks are the one suite where Flavor 6 clearly exceeds Flavor 2 (+4.5pp), which is consistent with CLIP-space supervision preserving the semantic attribute, cube color under occlusion, that Permanence questions turn on.

Table 4.5 Suite Contribution to Overall Accuracy, Flavors 2 and 6 (Test Split, Control=None). Decomposition of pooled Overall accuracy by suite. Pooled Acc. is item-weighted; Macro Acc. weights each suite’s available qtypes equally, isolating any effect of differing item counts across qtypes (Imitation lacks a constructable `task_goal` question). The close agreement between Pooled and Macro Acc. for every suite/flower confirms Imitation’s lower contribution is a genuine consequence of its smaller item count, not a distortion of its per-item quality. Contribution (pp) sums to each flavor’s Overall accuracy.

Suite	<i>n</i>	# Qtypes	Flavor 2		Flavor 6		Contrib. (pp)
			Pooled (%)	Macro (%)	Pooled (%)	Macro (%)	F2 / F6
Counting	1600	4	47.8	47.8	44.3	44.3	13.3 / 12.3
Permanence	1526	4	47.3	47.6	51.8	52.2	12.5 / 13.7
Reference	1450	4	44.1	43.7	40.6	40.3	11.1 / 10.2
Imitation	1186	3	50.4	50.2	48.1	48.0	10.4 / 9.9
Total	5762	–	–	–	–	–	47.2 / 46.1

Taken together, the results of this chapter establish projected NIR weight delta tokens as a queryable episode representation for a language model, and the noise control ties that decoding to the episode itself: the margins on `choice_action`, `gripper`, and `subgoal` collapse to chance when the delta is replaced by noise, and they collapse only for configurations fitted from the shared origin. Of the two questions the ablation raised, the first is settled by this control, since the random-origin margin does not survive it and therefore reflects dataset statistics rather than memory. The second, whether feature supervision confers benefit beyond the language-adjacent labels, is returned to in Chapter 6. The next chapter asks whether the same weight-space memory also supports prospective planning, conditioning the robot’s next action on the encoded history.

Chapter 5

Projecting NIR Weights into Action Space

Chapter 4 read the weight-space memory retrospectively. This chapter takes up the prospective question it closed on, whether the same representation can condition what the robot does next. The shift is in both direction and in measurement, as the labels in this work thus far has been categorical, so accuracy can register that information is present without measuring how tightly it constrains a continuous quantity. This chapter explores this gap by projecting the same memory representation of Flavor 2, the best performing configuration of Chapter 4, into action space. A vision-language-action (VLA) architecture (Brohan et al., 2023; Wen et al., 2025) whose vision input is replaced by the weight-space memory Δ of an NIR episode, a configuration we refer to as a memory-language-action (MLA) system, trained to regress the continuous actions RoboMME records alongside each episode (Figure 5.1).

5.1 Task Formulation and Scope

RoboMME’s per-timestep action fields (Section 3.2.1) supply the target: `joint_action`, the absolute joint-space command, shape (8,), holding the seven target joint angles of the Franka Panda arm and a binary gripper directive. It is read out at the cut frame T_{cut} , as the configuration commanded at the point where the NIR’s observation ends and a directive for what the arm should do next. Extending the formulation of Section 3.1, in which the answer is

$$\hat{y} = g(f_{\psi}(\Delta), q), \quad (5.1)$$

the linguistic decoding of g ’s output is replaced by a regression head h_{ω} applied to its final hidden state:

$$\hat{a} = h_{\omega}(g(f_{\psi}(\Delta), q)) \approx a \in \mathbb{R}^8, \quad (5.2)$$

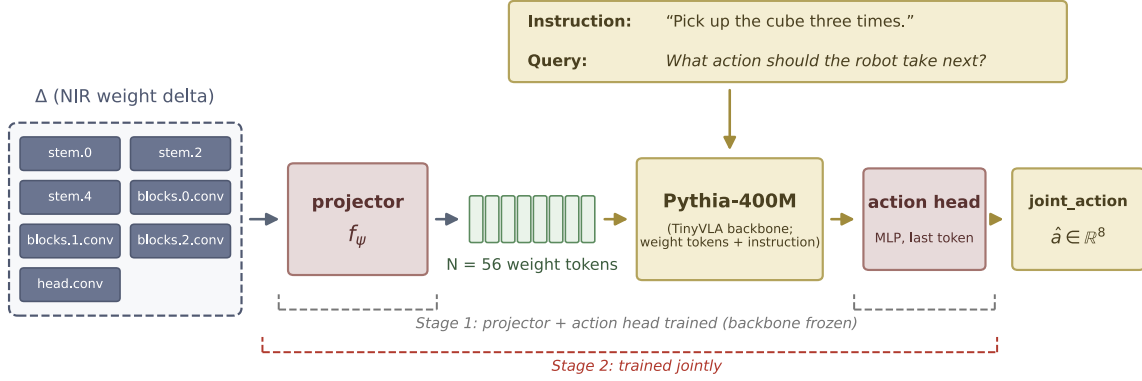


Fig. 5.1 **Chapter Overview.** The projection pipeline of this chapter. Weight delta Δ (one tensor per NIR layer) is projected into $N = 56$ weight tokens and spliced into the pretrained TinyVLA backbone (Pythia-400M) alongside the task instruction, and an action head regresses the eight-dimensional joint_action from the final token’s representation. Stage 1 trains the projector and action head with the backbone frozen; Stage 2 fine-tunes all components jointly. No video frame is seen at any point.

with g instantiated here by a pretrained TinyVLA (2025) backbone, and q by the given episode’s instruction. This is behavior cloning at a single decision point, with one demonstrated action per episode as the regression target instead of a per-timestep trajectory.

Unlike deployed VLA systems, which are closed-loop policies evaluated by rollout (Brohan et al., 2023), this is necessarily offline: the input does not exist until the RoboMME episode is over, so what is measured is action decoding from episodic memory rather than policy performance.

5.2 Methodology

The backbone is the pretrained Pythia-400M (GPT-NeoX) language model (Biderman et al., 2023) released with TinyVLA, with the vision tower, multimodal projector, and diffusion head discarded. This is due to no image being processed, and direct regression sufficing for this single target vector. Weight tokens are produced and injected exactly as in Section 4.1.1, with the question of Section 4.1.2 replaced by

```
<point>
Instruction: {task goal}
What action should the robot take next?
```

`<point>` expands into the fixed token span as done previously. The linguistic decoding is replaced by h_ω , a three-layer MLP, on the hidden state of the final non-padding position, which is the sole token that has attended to the entire weight-token span and instruction under causal masking. Targets are normalized per dimension, as raw scales differ by over an order of magnitude (Table 5.1). Training minimizes the Huber (smooth- ℓ_1) loss (Huber, 1992) on the normalized residual $r_d = \hat{a}_d - a_d$, applied per dimension and averaged,

$$\mathcal{L}_\delta(r_d) = \begin{cases} \frac{1}{2}r_d^2 & \text{if } |r_d| \leq \delta, \\ \delta(|r_d| - \frac{1}{2}\delta) & \text{otherwise,} \end{cases} \quad (5.3)$$

with $\delta = 1$.

5.3 Training and Evaluation Protocol

Training follows the two-stage scheme of Section 4.1.3, differing only in duration: Stage 2, which unfreezes the Pythia backbone, runs for 10 epochs rather than 3, as the regression head trains from scratch. Reported metrics are per-dimension R^2 over the 2,400 test episodes and MAE in raw units, with sign accuracy additionally reported for the binary gripper dimension.

Two controls mirror Sections 4.1.4 and 4.1.3: each checkpoint is re-evaluated with the episode’s delta replaced by Gaussian noise of matching shape before projection, so the noise passes through the trained projector with all else fixed, and R^2 is recomputed within each task, where the constant instruction can contribute nothing.

5.4 Results

Table 5.1 summarizes the central outcome. With the backbone frozen, the projector and action head alone reach a mean R^2 of 0.48. The stage 2 fine-tuning raises this to 0.51, and the improvement holds in all eight dimensions individually. Stage 2 also lowers the mean absolute error, to roughly 0.15 radians averaged over dimensions. Figure 5.2 presents this performance: pooled over the continuous joints, the prediction density concentrates along the identity line, and complete per-episode action profiles track the ground truth closely, with even episodes up to the 90th percentile of prediction error recovering the coarse shape of the true action.

Figure 5.3 resolves these aggregates into per-dimension structure. The strongly decoded joints (j1, j3, j5) form tight bands along the identity line, while j6’s scatter is visibly diffuse, matching its position in Table 5.1. Dense columns at recurring pose values, most visible for j5, correspond to reset configurations that the model reproduces. The gripper command takes only

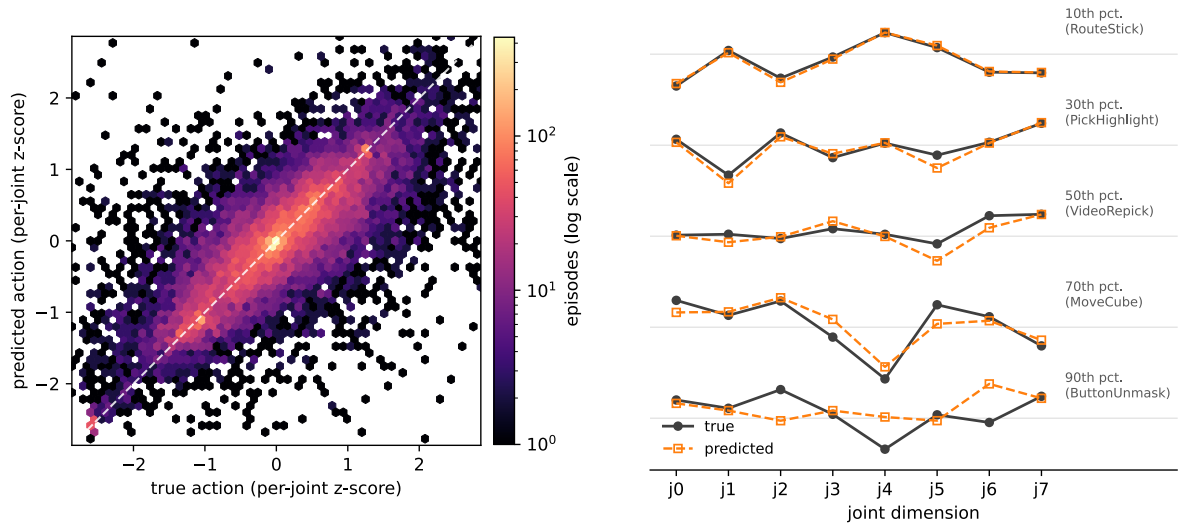


Fig. 5.2 **Decoding actions from weight-space memory.** Left: predicted against true actions pooled over the seven continuous joint dimensions, each z-scored by its own target statistics; the dashed line marks perfect prediction and colour indicates prediction density on a log scale. Pooled R^2 over these dimensions is +0.53. Right: complete predicted action profiles (orange) against ground truth (black) for episodes at fixed percentiles of mean absolute prediction error, selected mechanically by that ranking rather than by hand. The gripper dimension is excluded from the pooled panel and is predicted with 81.4% sign accuracy.

Table 5.1 **Offline action decoding on the test split (Flavor 2).** Per-dimension R^2 for both stages, with Stage 2 mean absolute error and the target standard deviation in raw units (radians; the gripper command j7 is binary, ± 1). Stage 2 improves on Stage 1 in every dimension. Mean R^2 is the unweighted average over the eight dimensions; bootstrap 95% intervals over test episodes are [0.459, 0.509] for Stage 1 and [0.488, 0.539] for Stage 2.

Dimension	Stage 1 R^2	Stage 2 R^2	Stage 2 MAE	Target SD
j0	0.40	0.43	0.066	0.140
j1	0.75	0.77	0.102	0.292
j2	0.43	0.44	0.047	0.100
j3	0.72	0.75	0.151	0.430
j4	0.30	0.34	0.031	0.066
j5	0.71	0.76	0.113	0.338
j6	0.17	0.20	0.263	0.470
j7 (grripper)	0.40	0.42	0.447	0.997
Mean	0.48	0.51	0.15	—

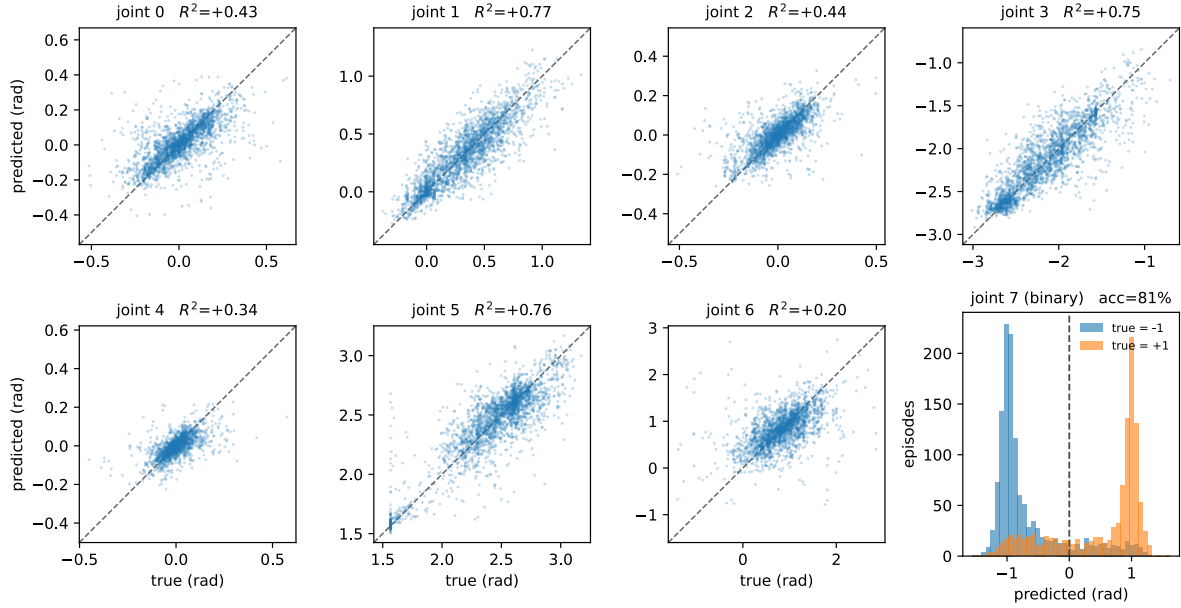


Fig. 5.3 **Per-dimension predictions on the test split (Stage 2).** For the continuous joints (j0 to j6), predicted against true values in raw units, with the identity line dashed and per-panel R^2 . The gripper command (j7) is binary (± 1), so its panel shows the distribution of the continuous output per true class with the decision threshold dashed; thresholding at zero yields 81.4% sign accuracy.

the values ± 1 , so its panel instead shows the distribution of the model’s continuous output for each true class: the two classes separate around the zero threshold, and the residual overlap accounts for the 18.6% of episodes whose sign is predicted incorrectly.

The controls validate the results. Substituting Gaussian noise for the delta before projection, with the instruction and every other aspect of evaluation held fixed, collapses the mean test R^2 from $+0.48$ to -0.39 for the Stage 1 model and from $+0.51$ to -0.55 for Stage 2, and drives every individual dimension below zero (Figure 5.4). The Stage 2 collapse reaches -1.15 on Joint 1. This verifies that the trained system conditions on the token span, producing wrong actions when the tokens are uninformative instead of retreating to the marginal mean. The within-task analysis points the same way. Mean R^2 remains positive in all sixteen tasks, spanning roughly 0.19 (InsertPeg) to 0.80 (StopCube) (Figure A.1), so the pooled result is not an artifact of the model separating tasks by their instructions. Some ordering structure is visible, with the unmasking tasks decoding best and the fine-manipulation and ordering tasks worst. A plausible reading is that the delta encodes coarse scene and pose structure more faithfully than fine manipulation detail, a pattern also seen in the language space results of Section 4.3.

Joint 6 is the weakest continuous dimension at $R^2 = 0.20$, and with its target standard deviation being the third largest in the table, its variation across episodes appears to be difficult

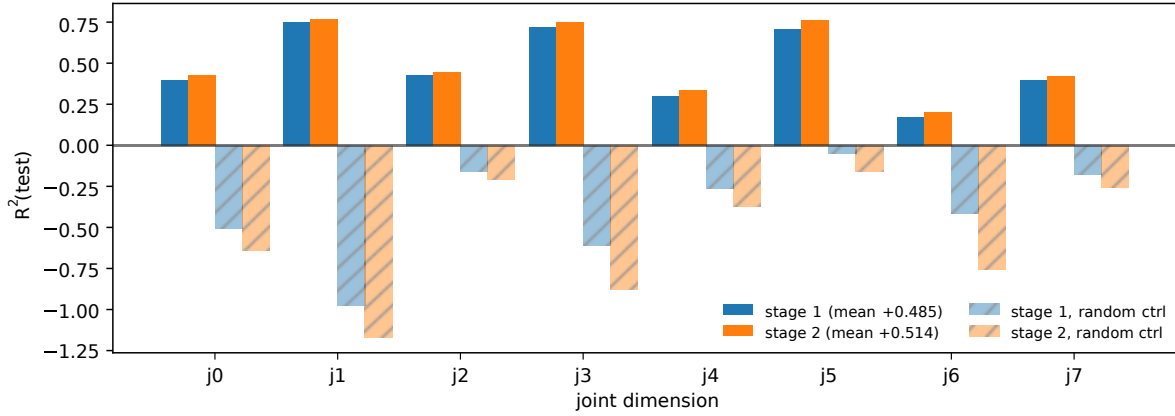


Fig. 5.4 **Random-input control (test split)**. Per-dimension R^2 for both stages with real weight tokens (solid) and with Gaussian noise substituted for the weight deltas before projection (hatched), under an identical evaluation. Real tokens yield positive R^2 in every dimension. The noise substitution drives every dimension below zero (Stage 1 mean -0.39 , Stage 2 mean -0.55).

to recover from the weight representation. The gripper behaves differently. Its R^2 of 0.42 understates performance on what is in fact a binary target: thresholding the continuous output yields 81.4% sign accuracy. This is the strongest gripper readout obtained thus far, exceeding both the Flavor 2 linear probe of Chapter 3 (66.2%) and the Stage 2 QA accuracy of Table 4.3 (69.1%). The three estimates come from different architectures and output spaces, yet their agreement on the presence of gripper information, together with the ordering, suggests that a nonlinear, instruction-conditioned system can extract structure that a linear map on the same representations does not.

Taken together with the QA results of Chapter 4, these findings extend the readout claim of this thesis beyond language. From a weight delta of an NIR episode, paired with the task instruction, the system predicts the robot’s next absolute joint command with a mean R^2 of 0.51 on held-out episodes, a mean error of roughly 0.15 radians, and the correct gripper decision in 81.4% of cases. Half of the across-episode variance in what the robot should do next is therefore recoverable from weight-space memory without a single frame. Chapter 4 showed that the episodic memory can be recalled, and this chapter shows it can be acted on.

Chapter 6

Discussion and Future Work

The hypothesis stated in Chapter 1 was that neural implicit representation weights constitute a semantically informative memory representation sufficient for non-Markovian downstream reasoning. To answer this, video episodes were encoded as NIR weight deltas fitted from a shared MAML meta-initialization, and the resulting deltas alone, with no frame access at any point, were interrogated in three ways: probed linearly for task-relevant semantic information, queried in natural language by a fine-tuned LLM, and paired with a pretrained VLA backbone to predict the robot’s next joint command. The main findings are as follows:

- Task-relevant information is linearly decodable from the raw weight deltas before any downstream model is involved, at up to $3.7\times$ the chance floor: action selection at 48.3% against 13.1% and subgoal identification at 29.9% against 8.0% (Table 3.4).
- Fifty-six projected weight tokens support above-chance question answering on every label type, with margins of +16.1, +19.1, +9.5, and +3.5 percentage points on `choice_action`, `gripper`, `subgoal`, and `task_goal`, at a pooled Cohen’s κ of 0.446 (Table 4.3).
- The same tokens, spliced into a pretrained VLA backbone, constrain the robot’s next joint command at a mean R^2 of 0.51 with 81.4% gripper sign accuracy and a mean error of roughly 0.15 radians, with positive R^2 inside all sixteen memory-intensive tasks (Table 5.1, Figure A.1).

Together, these results support the hypothesis, specifically that a memory representation encoded in NIR weight deltas is legible to a downstream model without frame re-access. The closest point of comparison in existing work, LLaNA (Amaduzzi et al., 2024), established that projecting NeRF weights into a language model supports reasoning over a static 3D scene, from that scene’s fully fitted weights. This work is novel on three counts: the encoded signal is a temporally extended, memory-intensive robot episode rather than a static scene, the

projected object is an adaptation delta from a shared meta-learned origin rather than a full set of independently fitted weights, and the application extends past language into continuous action decoding.

The remainder of this chapter synthesizes these findings and their significance (Section 6.1), states the limitations (Section 6.2), and closes with future work (Section 6.3).

6.1 Synthesis of Results

Two design factors produce the structure reported above, and the matched controls quantify how much of each configuration’s performance the episode itself contributes. The first is that the shared meta-learned origin is the decisive factor. Random-origin configurations sit at baseline under the probe (Table 3.4), and meta-initialization contributes the largest end-to-end improvement, 9.2 points overall (Table 4.4). The noise substitution expresses the same separation as a retained margin: meta-initialized, pixel-supervised flavors owe 8 to 11 points of overall accuracy to the real delta, against under 3 for every random-origin or pixel-free configuration (Table 4.4, Δ_{noise} column), and in action space the real weight tokens produce positive R^2 in all eight dimensions where their substituted counterparts fall below zero (Figure 5.4). The control is equally diagnostic of the random-origin configurations: their above-baseline accuracy survives the substitution nearly unchanged, so what they appear to recover derives from question and candidate statistics absorbed by the fine-tuned language model rather than from anything their weights encode. A margin over chance is therefore attributable to memory only where the control certifies it, and only the meta-initialized, pixel-supervised flavors pass that test.

The second observation is, the pixel reconstruction objective is what writes episode-specific content into the weights, and it performs representational work that fidelity does not measure. Flavor 3, which keeps the shared origin but abandons reconstruction, probes below baseline on three of four labels and lands beneath even random-origin Flavor 1 end-to-end (Tables 3.4 and 4.4). Feature-only supervision under-constrains the fit: a single pooled 768-dimension vector per frame is satisfied by many weight configurations, none required to commit to the episode’s specific appearance, whereas full-frame reconstruction ties every output unit to observed pixels at every timestep. The fidelity dissociation discussed in 3.4.3 supports this, since mean PSNR varies by under 2 dB across pixel-supervised flavors while probe gain varies from nil to several multiples of chance.

The definitional demand of Section 2.1 is thereby met. Weight deltas support retrospective recall and prospective action decoding, each verified by its own control, and was consumed by one probe and two different pretrained backbones without re-encoding the video. The weight

delta is a persistent and individually addressable memory object, a property no activation-bound representation offers.

These findings bear directly on the research need identified in Chapter 1. Existing memory mechanisms either retain raw observations for retrieval (Lewis et al., 2020), hold information in activations that expire with the forward pass (Vaswani et al., 2017), or, in the parametric case, maintain a single continually overwritten store developed for language streams, such as Titans (Behrouz et al., 2024) and GradMem (Kuratov et al., 2026). The results answer both questions that line of work left open in Section 2.1: the parametric principle transfers to non-Markovian visual episodes, and separable per-episode traces are viable. This also positions weight-space traces as a complement to Titans’ running memories wherever individually addressable episodes matter.

The results also carry evidential weight for the treatment of network parameters as a data modality (Dupont, Kim, et al., 2022; Han et al., 2026). Work in this area has asked what weights reveal about the model that produced them, or has decoded fitted implicit representations of static signals through classifiers and encoders (Gielisse & van Gemert, 2025; Luigi et al., 2023; Zhou et al., 2023). The prediction that semantic information in fitted weights depends on adaptation from a shared origin, has until now been supported architecturally, since Functa’s modulations exist only relative to a base network (Dupont, Kim, et al., 2022), adopted as fitting practice for efficiency and geometric convenience (Dupont, Loya, et al., 2022), and argued theoretically through the implicit function theorem (Qiu et al., 2026). The initialization ablation of Chapter 4, in which random-origin configurations fitted under identical supervision retain no margin once the noise control removes episode content, provides direct empirical evidence that the shared origin is invaluable to making NIR weight information consistently readable across fitted networks. This extends prior architectural and theoretical arguments for shared-origin weight spaces to temporally extended signals at the scale of foundation model decoders.

Finally, for robotics, where RoboMME identifies memory as a binding constraint on generalist policies (Dai et al., 2026), the action-space result suggests memory representations, rather than frame retention alone, as a design axis for vision-language-action systems (Wen et al., 2025).

6.2 Limitations

Several limitations bound these conclusions. Starting from a statistical standpoint: all accuracies rest on two training seeds, so differences of one to three points between adjacent configurations should not be over-read. Scale is another limitation, with the corpus comprising 1,600 source episodes, which is $25\times$ smaller than the 40,000 NeRF objects on which the related work of

LLaNA (Amaduzzi et al., 2024) trains its projector. This work can benefit from more data than the RoboMME benchmark supplies.

A second group of limitations surround the capacities of what the NIR weight representations can encode. Frames are rendered at 64×64 pixels, and the capacity-length tradeoff of Section 3.4.1 shows fidelity degrading on the longest episodes, so fine visual attributes may not survive encoding at this resolution. In terms of reconstruction quality, only PSNR was measured for the fitted NIRs without other image quality evaluations. Additionally, each frame in a RoboMME episode is dominated by the static tabletop, with the arm and manipulated objects occupying only a fraction of the image (see Figure 3.2), the content most relevant to the question types may also receive the least of the NIR’s capacity. A further consideration is the compute-storage trade-off of constructing the memory. Each episode is encoded into approximately 1.26M parameters through 500 fitting steps, while frame-based approaches instead incur costs from storing, retrieving, and re-encoding visual observations. These trade-offs are not compared directly in this work.

Several design choices were held fixed rather than explored. The projector f_ψ pools each tensor’s token axis to `tokens_per_layer = 8` throughout. Whether a larger budget of 16 or 32 tokens would alter the results is untested. Likewise, the auxiliary objectives examined, DINO and CLIP feature distillation, did not greatly improve on pixel reconstruction or QA, and no alternative objective was investigated in their place.

The evaluation is also bounded in scope. Scoring a closed candidate set by likelihood establishes that the weight-tokens support reliable discrimination, but does not establish that the model can construct free-form answers from the same information, which remains untested. The language model is a single 1.1B-parameter backbone, so whether the projector interface transfers across model families is unknown. Another limitation is that the memory construction is offline: this does not yet constitute an online memory that evolves with time, such as in the work of Titans (Behrouz et al., 2024).

Lastly, no external baseline, such as a vision-language model given the observed frames directly or a retrieval system retaining selected frames, is evaluated against the proposed pipeline. The experiments therefore establish that weight-space memories contain usable information, but do not determine the relative predictive or computational trade-offs between encoding an episode once into parameters and retaining or repeatedly encoding its original visual observations.

6.3 Future Work

Several extensions follow directly from the limitations and findings of this work. Scaling NIR capacity with episode length, and fitting at a higher rendering resolution than the current 64×64 , would test whether the PSNR drop on certain episodes, such as the long Reference-suite episodes discussed in Section 3.4.1 is a fixed-capacity artifact rather than a property of the representation itself. Following that, measuring SSIM (Structural Similarity Index Measure) and LPIPS (Learned Perceptual Image Patch Similarity) alongside PSNR would aid in a stronger verification of NIR fitting quality. Evaluating open-ended generation against the same four labels, scored by exact match or principled methods such as ROUGE-L rather than by choice-scoring, would test whether the discriminative signal measured here survives when the model can no longer rely on eliminating implausible candidates from a shown set. Fitting the NIR in real time, as frames arrive, rather than offline over a complete episode, would test whether the representation remains useful in deployment. Since neither DINO nor CLIP supervision largely improved end-to-end performance, other auxiliary objectives are worth testing for whether they encode information these two could not. Finally, probing the model’s ability to localize a state to a specific point in time, e.g. whether the gripper was open or closed at frame $N/2$ of an N -frame episode, rather than at the episode’s end (here, in T_{cut}) would test whether the weight tokens support temporal localization.

The tabletop-dominated composition of RoboMME frames suggests a NIR architecture such as ActINR (Kayabasi et al., 2025), which separates shared scene content from frame-specific content across distinct implicit networks, letting the delta spend its capacity on what changes between frames rather than on the static background. The offline construction of memory, meanwhile, marks the distance between this pipeline and continually updated parametric memory of the kind Titans (Behrouz et al., 2024) implements: each cut here is fit independently from the shared origin θ , with no information carried from one cut to the next. Warm-starting each cut’s fit from the immediately preceding cut’s weights, rather than from θ directly, would test a streaming variant in which the memory evolves incrementally as frames arrive, while a probe repeated at each cut would show whether the resulting weights retain the separable, per-episode traces that make the current offline design legible to a linear readout.

References

- Amaduzzi, A., Ramirez, P. Z., Lisanti, G., Salti, S., & Stefano, L. D. (2024). Llana: Large language and nerf assistant. <https://arxiv.org/abs/2406.11840>
- Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*.
- Behrouz, A., Zhong, P., & Mirrokni, V. (2024). Titans: Learning to memorize at test time. <https://arxiv.org/abs/2501.00663>
- Biderman, S., Schoelkopf, H., Anthony, Q., Bradley, H., O'Brien, K., Hallahan, E., Khan, M. A., Purohit, S., Prashanth, U. S., Raff, E., Skowron, A., Sutawika, L., & van der Wal, O. (2023). Pythia: A suite for analyzing large language models across training and scaling. <https://arxiv.org/abs/2304.01373>
- Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., Choromanski, K., Ding, T., Driess, D., Dubey, A., Finn, C., Florence, P., Fu, C., Arenas, M. G., Gopalakrishnan, K., Han, K., Hausman, K., Herzog, A., Hsu, J., Ichter, B., ... Zitkovich, B. (2023). Rt-2: Vision-language-action models transfer web knowledge to robotic control. <https://arxiv.org/abs/2307.15818>
- Chen, H., He, B., Wang, H., Ren, Y., Lim, S.-N., & Shrivastava, A. (2021). Nerv: Neural representations for videos. <https://arxiv.org/abs/2110.13903>
- Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 37–46. <https://doi.org/10.1177/001316446002000104>
- Dai, Y., Fu, H., Lee, J., Liu, Y., Zhang, H., Yang, J., Finn, C., Fazeli, N., & Chai, J. (2026). Robomme: Benchmarking and understanding memory for robotic generalist policies. <https://arxiv.org/abs/2603.04639>
- Dupont, E., Kim, H., Eslami, S. M. A., Rezende, D., & Rosenbaum, D. (2022). From data to functa: Your data point is a function and you can treat it like one. <https://arxiv.org/abs/2201.12204>
- Dupont, E., Loya, H., Alizadeh, M., Goliński, A., Teh, Y. W., & Doucet, A. (2022). Coin++: Neural compression across modalities. <https://arxiv.org/abs/2201.12904>

- Finn, C., Abbeel, P., & Levine, S. (2017). Model-agnostic meta-learning for fast adaptation of deep networks. <https://arxiv.org/abs/1703.03400>
- Gielisse, A., & van Gemert, J. (2025). End-to-end implicit neural representations for classification. <https://arxiv.org/abs/2503.18123>
- Graves, A., Wayne, G., & Danihelka, I. (2014). Neural turing machines. <https://arxiv.org/abs/1410.5401>
- Gwilliam, M., Zhang, R., Padmanabhan, N., Du, H., & Shrivastava, A. (2025). How to design and train your implicit neural representation for video compression. <https://arxiv.org/abs/2506.24127>
- Han, X., Wang, Z., Zhao, B., Zhang, B., Li, J., Borth, D., Yu, R., Maron, H., Ye, Y., Yin, L., & Neri, F. (2026). A survey of weight space learning: Understanding, representation, and generation. <https://arxiv.org/abs/2603.10090>
- Hendrycks, D., & Gimpel, K. (2016). Gaussian error linear units (gelus). <https://arxiv.org/abs/1606.08415>
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Comput.*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Huber, P. J. (1992). Robust estimation of a location parameter. In S. Kotz & N. L. Johnson (Eds.), *Breakthroughs in statistics: Methodology and distribution* (pp. 492–518). Springer New York. https://doi.org/10.1007/978-1-4612-4380-9_35
- Kayabasi, A., Vadathya, A. K., Balakrishnan, G., & Saragadam, V. (2025). Bias for action: Video implicit neural representations with bias modulation. <https://arxiv.org/abs/2501.09277>
- Kerr, J., Kim, C. M., Goldberg, K., Kanazawa, A., & Tancik, M. (2023). Lerf: Language embedded radiance fields. <https://arxiv.org/abs/2303.09553>
- Kingma, D. P., & Ba, J. (2017). Adam: A method for stochastic optimization. <https://arxiv.org/abs/1412.6980>
- Kurатов, Y., Kairov, M., Bulatov, A., Rodkin, I., & Burtsev, M. (2026). Gradmem: Learning to write context into memory with test-time gradient descent. <https://arxiv.org/abs/2603.13875>
- Landis, J., & Koch, G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1), 159–174. <https://doi.org/10.2307/2529310>
- LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4), 541–551.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. <https://arxiv.org/abs/2005.11401>

- Liu, H., Li, C., Wu, Q., & Lee, Y. J. (2023). Visual instruction tuning. <https://arxiv.org/abs/2304.08485>
- Liu, Y., Kothari, P., van Delft, B., Bellot-Gurlet, B., Mordan, T., & Alahi, A. (2021). Ttt++: When does self-supervised test-time training fail or thrive? *Proceedings of the 35th International Conference on Neural Information Processing Systems*.
- Luigi, L. D., Cardace, A., Spezialetti, R., Ramirez, P. Z., Salti, S., & Stefano, L. D. (2023). Deep learning on implicit neural representations of shapes. <https://arxiv.org/abs/2302.05438>
- Maiya, S. R., Girish, S., Ehrlich, M., Wang, H., Lee, K. S., Poirson, P., Wu, P., Wang, C., & Shrivastava, A. (2022). Nirvana: Neural implicit representations of videos with adaptive networks and autoregressive patch-wise modeling. <https://arxiv.org/abs/2212.14593>
- Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., & Ng, R. (2020). Nerf: Representing scenes as neural radiance fields for view synthesis. <https://arxiv.org/abs/2003.08934>
- Mu, T., Ling, Z., Xiang, F., Yang, D., Li, X., Tao, S., Huang, Z., Jia, Z., & Su, H. (2021). Maniskill: Generalizable manipulation skill benchmark with large-scale demonstrations. <https://arxiv.org/abs/2107.14483>
- Qiu, T., Sonis, C., & Shen, H. (2026). Ensuring semantics in weights of implicit neural representations through the implicit function theorem. <https://arxiv.org/abs/2601.23181>
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. <https://arxiv.org/abs/2103.00020>
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408. <https://doi.org/10.1037/h0042519>
- Shi, W., Caballero, J., Huszár, F., Totz, J., Aitken, A. P., Bishop, R., Rueckert, D., & Wang, Z. (2016). Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network. <https://arxiv.org/abs/1609.05158>
- Siméoni, O., Vo, H. V., Seitzer, M., Baldassarre, F., Oquab, M., Jose, C., Khalidov, V., Szafraniec, M., Yi, S., Ramamonjisoa, M., Massa, F., Haziza, D., Wehrstedt, L., Wang, J., Darcet, T., Moutakanni, T., Sentana, L., Roberts, C., Vedaldi, A., . . . Bojanowski, P. (2025). Dinov3. <https://arxiv.org/abs/2508.10104>
- Sitzmann, V., Chan, E. R., Tucker, R., Snavely, N., & Wetzstein, G. (2020). Metasdf: Meta-learning signed distance functions. <https://arxiv.org/abs/2006.09662>
- Sitzmann, V., Martel, J. N. P., Bergman, A. W., Lindell, D. B., & Wetzstein, G. (2020). Implicit neural representations with periodic activation functions. <https://arxiv.org/abs/2006.09661>

- Sun, Y., Wang, X., Liu, Z., Miller, J., Efros, A. A., & Hardt, M. (2020). Test-time training with self-supervision for generalization under distribution shifts. <https://arxiv.org/abs/1909.13231>
- van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(86), 2579–2605. <http://jmlr.org/papers/v9/vandermaaten08a.html>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. <https://arxiv.org/abs/1706.03762>
- Wang, R., Sun, Y., Tandon, A., Gandelsman, Y., Chen, X., Efros, A. A., & Wang, X. (2025). Test-time training on video streams. <https://arxiv.org/abs/2307.05014>
- Wang, Z., Bovik, A., Sheikh, H., & Simoncelli, E. (2004). Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4), 600–612. <https://doi.org/10.1109/TIP.2003.819861>
- Wen, J., Zhu, Y., Li, J., Zhu, M., Wu, K., Xu, Z., Liu, N., Cheng, R., Shen, C., Peng, Y., Feng, F., & Tang, J. (2025). Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. <https://arxiv.org/abs/2409.12514>
- Weston, J., Chopra, S., & Bordes, A. (2015). Memory networks. <https://arxiv.org/abs/1410.3916>
- Zhang, P., Zeng, G., Wang, T., & Lu, W. (2024). Tinyllama: An open-source small language model. <https://arxiv.org/abs/2401.02385>
- Zhou, A., Yang, K., Jiang, Y., Burns, K., Xu, W., Sokota, S., Kolter, J. Z., & Finn, C. (2023). Neural functional transformers. <https://arxiv.org/abs/2305.13546>

Appendix A

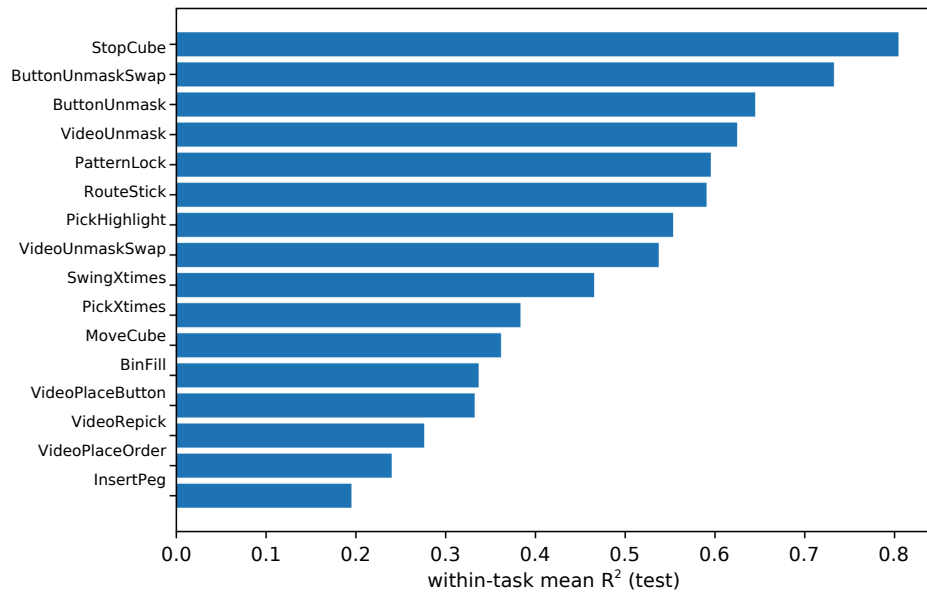


Fig. A.1 **Within-task action decoding on the test split (Stage 2).** Mean R^2 computed separately inside each of the sixteen RoboMME tasks, where the instruction is constant and task identity can contribute nothing to the score. All sixteen tasks remain above zero, from roughly 0.19 (InsertPeg) to 0.80 (StopCube).